

Lógica de Programação Algorítmica

José Fernando Marques Teixeira



Lógica de Programação Algorítmica





© by Ser Educacional

Todos os direitos reservados. Nenhuma parte desta publicação poderá ser reproduzida ou transmitida de qualquer modo ou por qualquer outro meio, eletrônico ou mecânico, incluindo fotocópia, gravação ou qualquer outro tipo de sistema de armazenamento e transmissão de informação, sem prévia autorização, por escrito, do Grupo Ser Educacional.

Imagens e Ícones: ©Shutterstock, ©Freepik, ©Unsplash.

Diretor de EAD: Enzo Moreira.

Gerente de design instrucional: Paulo Kazuo Kato.

Coordenadora de projetos EAD: Jennifer dos Santos Sousa.

Núcleo de Educação à Distância - NEAD Núcleo de Educação à Distância - NEA

José Fernando Marques Teixeira

Lógica de Programação Algorítmica:

Recife: Editora - 2023.

150 p.: pdf

ISBN: 978-65-5487-125-9

Palavras chave: 1. Lógica 2. Programação 3. Algoritmo

Grupo Ser Educacional
Rua Treze de Maio, 254 - Santo Amaro
CEP: 50100-160, Recife - PE
PABX: (81) 3413-4611
E-mail: sereducacional@sereducacional.com



Iconografia

Estes ícones irão aparecer ao longo de sua leitura:

	ACESSE Links que complementam o conteúdo.		IMPORTANTE Informações importantes que merecem atenção.
	ASSISTA Recomendação de vídeos e videoaulas.		OBJETIVO Descrição do conteúdo abordado.
	ATENÇÃO Informações importantes que merecem maior atenção.		OBSERVAÇÃO Nota sobre uma informação.
	CURIOSIDADES Informações interessantes e relevantes.		PALAVRAS DO PROFESSOR/AUTOR Nota pessoal e particular do autor.
	CONTEXTUALIZANDO Contextualização sobre o tema abordado.		PODCAST Recomendação de podcasts.
	DEFINIÇÃO Definição sobre o tema abordado.		REFLITA Convide a reflexão sobre um determinado texto.
	DICA Dicas interessantes sobre o tema abordado.		RESUMINDO Um resumo sobre o que foi visto no conteúdo.
	EXEMPLIFICANDO Exemplos e explicações para melhor absorção do tema.		SAIBA MAIS Informações extras sobre o conteúdo.
	EXEMPLO Exemplos sobre o tema abordado.		SINTETIZANDO Uma síntese sobre o conteúdo estudado.
	FIQUE DE OLHO Informações que merecem relevância.		VOCÊ SABIA? Informações complementares.



SUMÁRIO

Unidade 1

Apresentação e introdução a algoritmos	12
Utilizando a lógica	13
Uso de conectivos e tabelas-verdade	14
Conceitos iniciais sobre os algoritmos.....	17
Algoritmos: variáveis, operadores e constantes	22
Tipos de dados, declaração, atribuição e inicialização e variáveis ...	24
Conceito de operadores	27
Constantes: exemplo prático	30
Algoritmos: estruturas de decisão	34
Estrutura SE/ENTÃO.....	35
Estrutura SE/ENTÃO/CASO CONTRÁRIO.....	38
Estruturas SE/ENTÃO Aninhadas.....	40
Estrutura FAÇA CASO.....	42

Unidade 2

Visualg: fundamentos.....	48
Estruturas de repetição	56
Laço com repetição predefinida	57
Código em pseudolinguagem.....	58
Repetição com teste no início	60
Repetição com teste no fim.....	65
Visualg: estrutura de repetição	65



Para...faça	65
Enquanto...faça	67
Repita...até	68
Vetores e matrizes	70

Unidade 3

Matrizes.....	80
Inicializando matrizes.....	81
Matrizes bidimensionais	82
Matrizes tridimensionais.....	82
Classificação dos elementos.....	86
VisualG: matrizes	91
Preenchimento de matrizes com valores via teclado.....	93
Leitura de matriz e demonstração dos pares.....	94
Criação de uma matriz identidade de terceira ordem.....	96
Preenchimento de matriz de quarta ordem.....	98
Modularização de algoritmos	101
Vantagens do uso da modularização	102
Procedimentos	102
Funções	104
VisualG: Modularização	109
Passagem de parâmetros.....	112
Verificação dos termos	114

Unidade 4

Processo de programação	118
--------------------------------------	------------



Técnicas de desenvolvimento de programas.	118
Processo de software.	126
Linguagem de programação.	132
Características das linguagens de programação.	133
Aspectos fundamentais das linguagens de programação.	137
Classes de linguagem.	141
Estilo de codificação.	145



Apresentação

A lógica de programação é uma ciência utilizada para desenvolver programas de computadores e aplicativos de maneira geral e, assim, solucionar problemas que aparecem tanto na vida profissional quanto pessoal dos seres humanos. A lógica de programação algorítmica oferece a possibilidade de dividir grandes problemas em pequenas partes, com a finalidade de propor soluções e criar inovações tecnológicas.

É sobre esse relevante tema que estudaremos agora, começando pela introdução do conceito de algoritmo, que nada mais é que a solução de determinada demanda por meio de uma sequência lógica de instruções encadeadas (passo a passo). Neste conteúdo, iremos abordar também o conceito de pseudocódigos, e como essa técnica, amplamente utilizada na área de desenvolvimento de software, pode auxiliar futuros programadores a entender a dinâmica para desenvolver softwares e aplicativos.

A seguir, iremos tratar das características introdutórias à programação de computadores com o auxílio de um Ambiente Integrado de Desenvolvimento (IDE), o VisualG, e analisaremos decisões que podem ser tomadas durante a execução do programa, propondo soluções diferentes de acordo com comparações feitas após inseridos os valores através de variáveis.

Ainda abordaremos matrizes, modularização de algoritmos, processo e linguagens de programação e outros assuntos indispensáveis para a formação na área.

Bons estudos!





Autoria

José Fernando Marques Teixeira

O professor **José Fernando Marques Teixeira** graduado em Processamento Dados pela Universidade Bandeirante de São Paulo – UNIBAN (2001) e tem especialização em Análise de Sistemas pela Pontifícia Universidade Católica de Campinas – PUC (2003).

Com mais de 35 anos de atuação na área de Informática, já teve experiência com projetos de rede e laboratórios de Informática para escolas e universidades, ensino de Informática e coordenação de laboratórios de Informática em instituições de ensino.

Atualmente, atua como analista de suporte e coordenador de equipe, responsável pela área de TI, é palestrante em eventos e ministra cursos de Informática para funcionários em empresas e concessionárias em São Paulo.



Objetivos

1. Apresentar algoritmos e conceitos de lógica, evidenciando seus aspectos principais;
2. Abordar os tipos de algoritmo, debatendo aspectos referentes às variáveis, operadores e constantes;
3. Observar os principais conceitos referentes à estrutura de decisão, verificando seus comandos e atividades essenciais;



Apresentação e introdução a algoritmos

Para compreender aspectos relacionados ao uso dos algoritmos, é necessário entender este tema, fundamental à lógica de programação, por meio de do uso de softwares, cada dia mais presentes no nosso cotidiano, como em nossos celulares e aparelhos de GPS.

A procura por softwares está em constante crescimento no mercado, e a quantidade de profissionais especializados no seu desenvolvimento é consideravelmente inferior a essa demanda. Diante desse cenário, milhares de pessoas vêm se interessando pela área, principalmente com o objetivo de aprender a programar.

O aprendizado da lógica de programação, nesse contexto, é essencial para os profissionais da área de desenvolvimento, pois serve de referência para que qualquer linguagem de programação seja compreendida. Certamente, o uso da lógica, nos diversos contextos de nossa vida, facilita a solução de problemas. Trazendo essa ideia para os programas de computador, iremos notar que o trabalho com softwares também é, em grande parte, dedicado à solução de problemas, a depender da lógica adotada pelo programador.

Vamos iniciar nossos estudos explorando conceitos relacionados à lógica e aos algoritmos. Certamente você, caro leitor, pode ainda se questionar qual a importância destes termos no ramo da computação. Sobre isso, podemos afirmar que eles são pilares básicos no desenvolvimento de linguagens de programação direcionadas aos computadores digitais.

Sabe-se ainda que qualquer tipo de problema que se converte em um programa necessita, dentre outros aspectos, ser mensurado e planejado. Não é recomendável, sendo até arriscado, iniciar a escrita de código de programação sem realizar a análise dos problemas. É com base nisso que analisaremos, ao longo desta unidade os aspectos introdutórios mais relevantes acerca dos algoritmos e da lógica de programação.



Utilizando a lógica

Você sabe o que é lógica? A que aspectos ela está relacionada? Bem, a lógica, segundo o autor Renato Soffner em seu livro *Algoritmos e programação em linguagem C*, de 2013 (p. 18), está ligada a um pensamento que envolve duas características básicas: a razão e ordem. Na Filosofia, por exemplo, ela estuda o método que explica a maneira como pensamos, ou seja, nos induz a utilizar de maneira correta as leis que norteiam o pensamento.

De forma similar às outras ciências, a computação adota métodos matemáticos para estabelecer fatores com maior nível de precisão. Além disso, insere uma notação capaz de assegurar abstrações mais assertivas e raciocínios com maior rigidez. O grande objetivo da lógica de programação, assim, é assegurar que nós, na condição de usuários de um sistema, possamos entender o funcionamento das linguagens e das técnicas de programação.

É importante frisar que a lógica é responsável pela organização metódica do pensamento e das ações realizadas através do raciocínio. Basta que observemos, por exemplo, nossas formas de expressão: normalmente, usamos interrogações ou afirmações, levando em consideração aquilo que de fato queremos expressar. Porém, estas ações podem trazer dúvida ou imprecisão.

Um computador, por característica, não pode apresentar incerteza nas suas ações, por isso, é preciso utilizar ferramentas e técnicas lógicas para realizar a programação. Isso pode ser realizado através de sentenças (proposições) classificadoras como **verdadeiras** ou **falsas**. Vamos observar este conceito através das expressões abaixo:

- a. 20 é maior que 18
- b. Como você está?
- c. Fulano é muito rápido.
- d. Existe vida além da morte.



Vamos à análise das expressões: a frase (a) é uma sentença que associa um valor lógico que, no caso, é verdadeiro; a expressão (b) é considerada uma pergunta, não podendo ser considerada uma sentença dentro da área da lógica, já que é impossível determinar um valor falso ou verdadeiro; (c) também não pode ser considerada uma sentença nesses termos, pois é impossível determinar um valor lógico para ele; por fim, a frase (d) pode ser considerada uma sentença definida como verdadeira ou falsa.

Uso de conectivos e tabelas-verdade

Nossas relações ou conversas não se restringem ao uso de sentenças simplificadas. Normalmente, inserimos conectivos, para estabelecer relações que constituem as chamadas “sentenças compostas”. Segundo o entendimento de Soffner (2013, p. 19), o valor lógico atribuído a uma expressão composta irá depender dos valores atribuídos a cada sentença que a constitui e dos conectivos adotados.

Vamos aos casos práticos: considere como exemplos de conectivos mais usuais os termos “e”, “ou” e “não”. Agora realizemos uma combinação de duas sentenças, como: coelhos são velozes / tartarugas são lentas. Adicionando-se um conectivo, teremos uma sentença resultante, considerada verdadeira: coelhos são velozes e tartarugas são lentas.

Dentro do conceito de lógica, vamos utilizar um símbolo $\wedge$ para indicar o conectivo lógico “e”, além das letras maiúsculas A e B para apontar as sentenças que constituem a sentença composta. Caso A e B sejam verdadeiras, a expressão $A \wedge B$ será denominada de **conjunção** de A e B, na qual esses termos serão denominados de fatores de expressão.

É possível sintetizar as combinações realizadas pelas conjunções através de um método denominado de tabela-verdade, que tem a função de auxiliar na sistematizada tomada de decisão do valor lógico oriundo da combinação de proposições. Na Tabela 1, é possível verificar, por exemplo, que a conjunção de duas sentenças só será verdadeira quando ambas forem verdadeiras. A combinação



de uma sentença verdadeira com uma falsa resulta automaticamente em uma sentença composta falsa.

Tabela 1. Tabela-verdade da conjunção (e)

A	B	$A \wedge B$
V	V	V
V	F	F
F	V	F
F	F	F

Fonte: SOFFNER, 2013, p. 20. (Adaptado).

O conectivo “ou” é representado por meio do símbolo $\vee$. Nesta condição, a expressão $A \vee B$ é denominada de disjunção de A e B, e suas relações estão organizadas na Tabela 2.

Tabela 2. Tabela-verdade da disjunção (ou)

A	B	$A \vee B$
V	V	V
V	F	V
F	V	V
F	F	F

Fonte: SOFFNER, 2013, p. 20. (Adaptado).

O conectivo de negação é representado pelo sinal $\sim$, que inverte o valor lógico da proposição.



Tabela 3. Tabela-verdade da negação (não)

A	$\sim A$
V	F
F	V

Fonte: SOFFNER, 2013, p. 20. (Adaptado).

As sentenças podem ainda apresentar uma combinação na forma condicional, utilizando para isso o conectivo composto “se... então...”, representada graficamente por meio da nomenclatura $A \rightarrow B$ (A implica B). Podemos notar aqui que o conectivo lógico evidenciado é o da implicação. Logo, na sentença “**se** há verão, **então** há calor”, o termo antecedente é “há verão”, que implica em uma consequência: há calor.

Tabela 4. Tabela-verdade da implicação (se então)

A	B	$A \rightarrow B$
V	V	V
V	F	F
F	V	V
F	F	V

Fonte: SOFFNER, 2013, p. 21. (Adaptado).

Logo, os conectivos lógicos estabelecem uma combinação entre duas condições; e as tabelas-verdade estabelecem o resultado dessas combinações.



Conceitos iniciais sobre os algoritmos

O que são algoritmos? Qual sua importância na Ciência da Computação, mais especificamente na lógica de programação? Essas questões começam a ser respondidas quando compreendemos que os algoritmos são considerados o centro da computação, pois a tarefa mais importante, no momento de escrever e desenvolver um programa, é programar um algoritmo que traga soluções e consiga ser repetido de forma indefinida.

Mas qual o caminho para que os algoritmos sejam de fato construídos? Renato Soffner, em sua obra de 2013 (p. 22) descreve que os procedimentos obedecem aos seguintes passos:

- a) Avaliação de problemas;
- b) Identificação da entrada de dados;
- c) Definição das alterações a serem realizadas pelo algoritmo;
- d) Identificação das saídas;
- e) Desenvolvimento dos algoritmos através do diagrama de blocos.

Algoritmos podem ser simbolizados graficamente através de símbolos padronizados (diagramas de blocos), como se observa na figura a seguir:



Figura 1. Simbologia para diagramas de blocos.



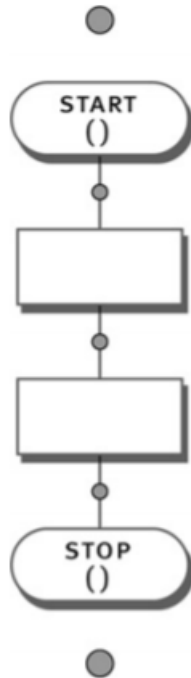
Fonte: SOFFNER, 2013, p. 22.

Outro aspecto que devemos tratar aqui é o fato de que qualquer programa direcionado a computadores pode ser redigido utilizando basicamente três modelos de estrutura de controle em um fluxo de programação: estrutura sequencial, de decisão e repetição. O uso dessas estruturas segue determinadas normas, com o objetivo



de impedir a introdução de um código mal elaborado e sem um controle sistematizado. Tais regras podem ser denominadas de programação estruturada. Podemos simbolizar essas representações dentro de diagramas de blocos dos elementos de programação estruturada, assegurando seu nível mínimo de eficácia (Diagrama 1).

Diagrama 1. Sequência Start (iniciar) / Stop (parar)



Fonte: SOFFNER, 2013, p. 23.

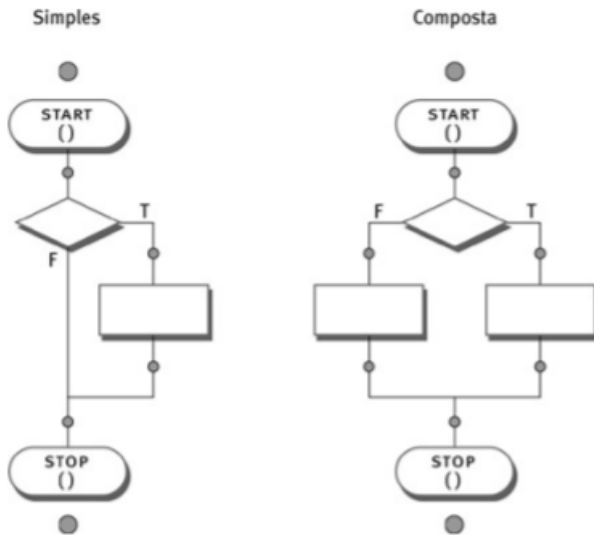
Esse algoritmo consegue assegurar que o programa seja realizado de maneira completa, sem se desvirtuar em caminhos desconexos. Isso representa um nível de unicidade das ações e processos realizados, levando em consideração a lógica idealizada pelo programador.

Quando nos referimos à decisão, que verificaremos com mais detalhes nos tópicos seguintes, podemos classificá-las como: simples, composta ou múltipla. A primeira é responsável pelo teste de



uma condição e, no caso de ser verdadeira, executar uma ação, sem a preocupação de executar uma atividade no caso de análise da condição oposta. Por sua vez, a decisão composta apresenta uma ação prevista no caso da observação de uma condição contrária, conforme se visualiza no Diagrama 2, a seguir.

Diagrama 2. Decisão simples e composta



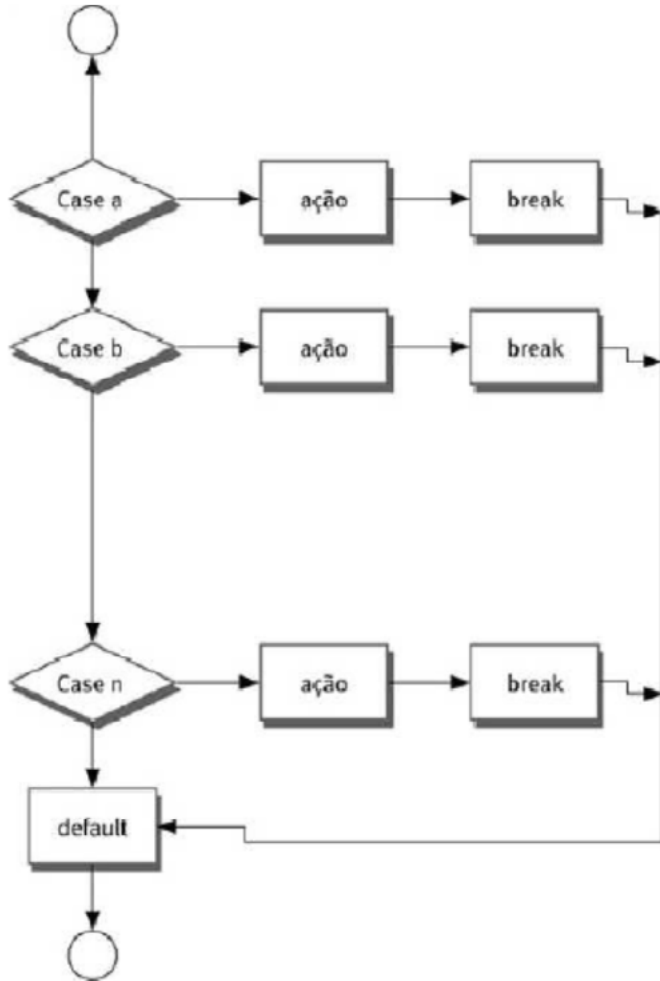
Fonte: SOFFNER, 2013, p. 24.

A decisão múltipla apresenta uma série de condições a serem avaliadas. Neste caso, é necessário uma estrutura mais robusta que a decisão composta, que se limita a analisar duas condições, apenas.

Outro detalhe importante é o alinhamento de decisões compostas, que serve como uma importante possibilidade de resolução de um problema: no caso da condição verdadeira, é possível realizar as ações pré-estabelecidas, dispensando o uso da estrutura e evitando a realização de um novo teste fora do contexto; em uma condição de falsidade, a possibilidade seguinte será testada e assim sucessivamente. Existe a possibilidade também de idealizar uma condição padronizada, na hipótese de os testes anteriores não terem verificados os valores de verdade, segundo o já mencionado livro de Renato Soffner.



Diagrama 3. Decisão múltipla



Fonte: SOFFNER, 2013, p. 25.

Outro tipo de estrutura de controle é a repetição com teste, que inicialmente analisa uma condição antes de executar as ações que se repetem e são previstas. Caso seja válida, o processamento vai entrar em iteração, até que a condição deixe de ser verdadeira, à medida que o programa segue normalmente para as rotinas



programadas restantes. Essa repetição é adequada, por exemplo, para executar testes de senhas anteriores ao acesso e funções em repetição no programa. Se a repetição com teste ocorrer, no fim, realizará uma atividade, ao menos uma vez, antes decidir sua continuação. É muito usada para realizar validações de entradas de dados no período anterior da sequência ao programa.

Algoritmos: variáveis, operadores e constantes

Durante os procedimentos realizados pelos algoritmos, é possível haver a necessidade de arquivar informações, de maneira temporária, que serão usadas durante o processamento. Trazendo esse conceito para a área da programação, os dados armazenados temporariamente ao longo da execução de um algoritmo são definidos como variáveis, segundo afirmam os autores Humberto Martins Beneduzzi e João Alberto Metz no livro *Lógica e linguagem de programação: introdução ao desenvolvimento de software*, de 2010 (p.16). Para compreendermos como as variáveis são utilizadas na prática, podemos imaginar, por exemplo, um algoritmo desenvolvido para mensurar o índice de colesterol de uma pessoa e, ao final, verificar se está acima, abaixo ou no nível adequado.



EXPLICANDO

Para dar continuidade ao procedimento, é preciso colher informações relacionadas ao nível de colesterol e armazená-las momentaneamente, até que o cálculo deste índice seja realizado.

De maneira simplista é possível afirmar que uma variável, segundo Humberto Beneduzzi e João Alberto Metz, em 2010 (p. 16), é um lugar onde se arquiva valores reconhecidos através de um único nome. Isso indica que, toda vez que desejarmos escrever (atribuir)



ou visualizar o valor arquivado dentro de uma variável, é necessário identificar o nome atribuído a ele.

Tecnicamente, a memória do computador pode estar disposta em compartimentos menores, cada um com sua localização estabelecida, em uma situação análoga a uma rua, onde cada residência apresenta uma numeração como forma de identificação. No momento em que desenvolvemos uma variável, a mensagem que transmitimos ao computador é a de que desejamos utilizar um endereço de memória capazes de arquivar valores, apresentando um nome para acessá-lo e servir de referência.

O **pseudódigo** pode ser compreendido como uma técnica mais formalizada e estruturada, já que possui determinadas regras, similares àquelas utilizadas pelas linguagens de programação, e se caracteriza por disponibilizar uma estruturação básica referente à linguagem de programação.

Segundo os autores Beneduzzi e Metz na já citada obra, ao declaramos as variáveis presentes no Pseudódigo, alguns padrões são utilizados. Certamente, você deve estar se questionando quais padrões são os mais importantes. É possível adotar a seguinte padronização:

- Nomes adotados pelas variáveis podem dispor somente de caracteres alfanuméricos e underscores (_). É vedado o uso de acentos, espaços, dentre outros símbolos;
- Não é possível iniciar uma variável com números;
- A nomeação das variáveis pode se constituir por letras minúsculas ou maiúsculas, entretanto, é mais adequado utilizar as letras minúsculas, por uma questão de conversão em boa parte das linguagens de programação. Outro aspecto a ser considerado é a união de palavras pela inserção de uma letra inicial maiúscula, ligando-a com a palavra anterior (por exemplo, em “dataNas”, para representar o termo data de nascimento);
- Variáveis com um nome composto utilizam o underscore (_) para separar as palavras;



- Variáveis são apresentadas em bloco próprio, no começo do algoritmo ou na sub-rotina das quais fazem parte.

Além disso, quando se declara variáveis em um algoritmo, é adequado usar nomes mais curtos e representativos. Em vez de “altura_mínima_solicitada”, pode-se utilizar a expressão “altura_min”.

Tipos de dados, declaração, atribuição e inicialização e variáveis

No momento em que declaramos uma variável, é preciso apontar o modelo de informação que queremos arquivar. Certamente, existe uma variedade de modelos de dados usuais em boa parte das linguagens de programação. Vamos verificar os tipos mais usuais:

- **Inteiro:** reúne um conjunto de números inteiros, positivos ou negativos, por exemplo, os números 4, 0, -4;
- **Real:** compreende um conjunto de números inteiros ou fracionários, positivos ou negativos, como os números 3, 0, -3, 2,5 etc.
- **Caractere:** armazena caracteres alfanuméricos. Também é conhecido como um tipo literal (string), normalmente simbolizado entre aspas, por exemplo, “Rua dois irmãos, 180”, “#” etc.;
- **Lógico:** engloba basicamente valores lógicos, normalmente dos tipos verdadeiro e falso (V, F).

Uma variável, para ser usada em um contexto relacionado a um algoritmo, precisa de uma declaração.



O que podemos entender como declaração?

Declaração se trata de uma definição de variável, sendo informados seu nome e tipo de informação capaz de arquivar. Ao declarar variáveis em um pseudocódigo, é possível adotar um padrão, que pode ser representado desta maneira:

<nome de variável>: <tipo de dado>;

É importante salientar que a declaração pode ser realizada no início do algoritmo, dentro de um bloco denominado de variáveis, conforme se visualiza na estrutura seguir, adaptada do livro *Lógica e linguagem de programação: introdução ao desenvolvimento de software*, de Humberto Beneduzzi e João Alberto Metz, de 2010, (p. 19).

[código]

[Exemplo de declaração de variáveis I]

Algoritmo ExemploVariáveis

Variáveis

nome: Caractere;

endereço: Caractere

altura: Real

peso: Real

telefone: Caractere: (Declaramos como caractere para permitir que o telefone seja digitado com formatação)

Início

...

Fim

[/código]

Também é possível reunir as variáveis, no mesmo modelo, em uma linha só, declarando todas juntas, como podemos ver a seguir, na estrutura adaptada do livro de Humberto Beneduzzi e João Alberto Metz (p. 19).



[código]

[Exemplo de declaração de variáveis II]

Algoritmo ExemploVariaveisAgrupadas

Variáveis

Nome, endereço, telefone: Caractere;

Altura, peso: Real;

Inicio

...

Fim

[/código]

Quando tratamos da atribuição e inicialização de variáveis, é preciso inicialmente analisar suas definições. A atribuição é a definição do valor da variável analisando de maneira técnica. Isto significa redigir uma informação nova no espaço da memória, identificada pelo nome atribuído à variável no instante da declaração. Ao atribuir um novo valor, é preciso adotar um padrão:

<nome da variável>: <valor>;

A inicialização se refere à atribuição de um valor inicial no começo do algoritmo, após o demarcador “início”. É recomendável inicializar todas as variáveis, mesmo aquelas que não apresentam valor definido. É importante também deixar claro que a inicialização de variáveis pode ser vista como uma prática boa, porém, não é uma regra. Sendo assim, algumas variáveis, por exemplo, são inicializadas de maneira automática e através de um valor padrão (default), levando em consideração o tipo de dado, conforme se visualiza no quadro a seguir.

Quadro 1. Tipo de variáveis e seus valores

Tipo de Variável	Valor default
Inteiro	0



Real	0
Caractere	“ ” (vazio)
Lógico	F (falso)

Conceito de operadores

Até o momento descrevemos bastante sobre a importância do armazenamento de informações nas variáveis. Entretanto, seria inútil adquirir estas informações se os programas não conseguissem realizar ações com elas, como realizar cálculos. Diante desse cenário, as linguagens de programação disponibilizam operadores que possibilitam a execução de operações com dados.

É possível verificar que os operadores podem ser divididos basicamente em quatro grupos. Veremos com mais detalhes cada um deles.

Operadores aritméticos

Assim como na matemática, os operadores aritméticos se caracterizam por possibilitar a efetuação de cálculos através de dados inseridos na memória. Por conta disso, eles são usados exclusivamente com valores aritméticos. No Quadro 2, a seguir, é possível associar os operadores aritméticos às principais linguagens de programação utilizadas.

Quadro 2. Operadores aritméticos

Operador	Descrição
+	Adição: efetua a soma entre dois números
-	Subtração: efetua a subtração entre dois números
*	Multiplicação: efetua a multiplicação entre dois números
/	Divisão decimal: efetua a divisão entre dois números e dá como resultado um número em ponto flutuante

Fonte: ALVES, 2014, p. 44. (Adaptado).



Existem outros operadores utilizados em algumas linguagens, como observamos a seguir.

Quadro 3. Operadores adicionais

Operador	Descrição
\	Divisão inteira: efetua a divisão entre dois números inteiros e dá como resultado um número inteiro, desprezando a parte fracionária, caso exista.
%	Resto de divisão: retorna o resto de uma divisão não exata entre dois números inteiros.

Fonte: ALVES, 2014, p. 44. (Adaptado).

Operadores de comparação

São conhecidos como operadores relacionais, pois possibilitam que dois valores, duas variáveis ou até mesmo uma variável com um valor estabeleçam uma relação. Resumidamente, é possível estabelecer uma comparação entre uma variável e uma constante.

Quadro 4. Operadores de comparação

Operador	Descrição
<	Menor que
>	Maior que
<=	Menor que ou igual a
>=	Maior que ou igual a
=	Igual a
<>	Diferente de

Fonte: ALVES, 2014, p. 44. (Adaptado).



Operadores lógicos

Assim como os operadores relacionais, os operadores lógicos são usados no desenvolvimento de expressões lógicas que normalmente resultam em um valor verdadeiro ou falso. Esses operadores possibilitam a análise de uma expressão lógica e podem retornar um valor verdadeiro ou falso. Vale lembrar que as linguagens de programação são definidas na álgebra booleana.

Quadro 5. Operadores de álgebra booleana

Operador	Descrição
E	Realiza a operação lógica E. Caso ambas as expressões sejam verdadeiras, o resultado será verdadeiro.
Ou	Realiza a operação lógica Ou. Caso ambas as expressões ou uma delas sejam verdadeiras, o resultado será verdadeiro.
Não	Realiza a negação de uma expressão lógica. Se ela for verdadeira, será transformada em falsa e vice-versa.

Fonte: ALVES, 2014, p. 45. (Adaptado).

Operadores de manipulação de bits

Determinadas linguagens apresentam operadores diferenciados, que possibilitam a manipulação de bits de um byte. Esses operadores são essenciais em programas que realizam conversões da modalidade analógica/digital e na criação de programas direcionados em microcontroladores, por exemplo. Podemos observar os principais:

- **Deslocamento de bits à direita:** locomove todos os bits do byte/palavra à direita, zerando o primeiro bit à esquerda.
- **Deslocamento de bits à esquerda:** locomove todos os bits do byte/palavra à esquerda, zerando o primeiro bit à direita.
- **Rotação de bits à direita:** locomove todos os bits do byte à direita, além de gravar o último bit à direita na primeira posição à esquerda.



- **Rotação de bits à esquerda:** locomove todos os bits do byte à esquerda, além de gravar o ultimo bit à esquerda na primeira posição à direita.
- **Operação binária E:** retorna um byte/palavra, em que cada byte é fruto da aplicação do operador E (And) em cada bit pertencente a outros bytes envolvidos na operação.
- **Operação binária Ou:** retorna um byte/palavra, em que cada byte é fruto da aplicação do operador Ou (Or) em cada bit pertencente a outros bytes envolvidos na operação.
- **Operação binária Ou Exclusivo:** retorna um byte/palavra, em que cada byte é fruto da aplicação do operador Ou Exclusivo (Xor) em cada bit pertencente a outros bytes envolvidos na operação.

Constantes: exemplo prático

Quando se trata de constantes, é preciso compreender de imediato que este termo exerce basicamente a mesma função das variáveis, isto é, armazenar valores no momento em que o algoritmo está sendo utilizado. Entretanto, as constantes se diferenciam das variáveis por conta de um outro aspecto: após a declaração, não pode haver alterações nos valores, ou seja, elas irão arquivar valores que não se alterarão durante a execução do algoritmo. É importante ressaltar que as constantes precisam ser declaradas no bloco próprio, durante a fase inicial do algoritmo.

No que se refere à nomenclatura, as regras presentes na declaração de variáveis também são seguidas com uma recomendação: a declaração das constantes usa letras maiúsculas como maneira de se distinguir das variáveis de forma mais simples durante a execução de um algoritmo. A separação ocorre através do caractere underscore (__) para o caso dos nomes compostos. Não se trata necessariamente de uma norma, mas é adotada pela maioria dos programadores.



Quadro 6. Nomenclaturas das constantes

Exemplo de nomes recomendados I	Outro exemplo
FORCA_ACELERACAO	Forca_aceleracao
PI	PI
OPERACAO_PADRAO	operacaoPadrao

Fonte: BENEDUZZI; METZ, 2014, p. 23. (Adaptado).

No exemplo a seguir, adaptado do já mencionado livro *Lógica e linguagem de programação: introdução ao desenvolvimento de software*, de Beneduzzi e Metz (p. 23), é possível observar um pseudo-código que apresenta uma declaração de constantes.

[código]

Algoritmo Exemplo

Constantes

PI: 3,01 =; { constante do tipo real}

MAX_GRAUS : = 90; { constante do tipo inteiro}

INSTITUCIONAL : = “Escola X”; { constante do tipo caractere}

RELATÓRIO_ATIVADO : = F; (constante do tipo lógico)

Variáveis

...

Início

...

Fim

[/código]



De maneira mais didática, é possível elaborar um exemplo que apresenta a utilização de variáveis dentro dos pseudocódigos. William Pereira Alves, em seu livro *Linguagem e lógica de programação*, de 2014 (p. 38), traz um exemplo de cálculo do volume de um cilindro. Para tanto, é necessário utilizar uma fórmula matemática para calcular o volume, representado pela expressão $V = \pi \cdot r^2 \cdot h$, onde r corresponde ao raio do cilindro, e h , o conceito de altura.

De imediato, é preciso descrever, por meio de textos, como o programa deve agir, demonstrando mensagens que solicitem a inserção do raio e da altura referente ao cilindro. Posteriormente, é preciso mensurar o valor do volume e, ao final, apresentar o resultado ao usuário. Além disso, o algoritmo utilizado em pseudolinguagem precisa iniciar com a identificação do programa e com a declaração inicial a seguir:

[código]

PROGRAMAVolumeCilindro

INÍCIO

[/código]

Segundo William Alves em 2014 (p. 38), ainda é preciso declarar as variáveis e constantes que serão adotadas pelo programa. Certamente, a ordem em que a declaração é feita não influencia. Porém, por questões de padrão, recomenda-se declarar as constantes em primeiro lugar. Os valores do raio, altura e volume são fracionários e contêm casas decimais, portanto, pode-se declarar essas variáveis na condição de ponto flutuante que apresenta uma precisão mais simples:



[código]

CONSTANTE

PI: 3,14

VARIÁVEL

SplVolume, SplRaioCilindro, SplAlturacilindro: SIMPLES;

[/código]

O procedimento seguinte é entrar com os dados direcionados para o raio e altura do cilindro. Na pseudolinguagem adotada, o termo LER será considerado como o comando de entrada junto à variável que recebe os dados dispostos entre parênteses. Por sua vez, o comando responsável pela impressão de dados na tela apresenta a nomenclatura ESCREVER junto do texto ou variável que se encontra entre parênteses. Isso pode ser observado através do fragmento do código do algoritmo. O algoritmo finalizado está disposto a seguir, adaptado do livro de William Alves, *Lógica de programação de computadores: ensino didático*, de 2010 (p. 39).

[código]

PROGRAMA VolumeCilindro

INÍCIO

CONSTANTE

PI: 3,14

VARIÁVEL

SplVolume, SplRaioCilindro, SplAlturacilindro:

SIMPLES;

ESCREVER (“Entre com o valor do raio do cilindro: “);



```
LER (spRaioCilindro);  
ESCREVER ("Entre com o valor da altura do cilindro: ");  
LER (spAlturaCilindro);  
splVolume =PI *(splRaioCilindro * splRaioCilindro *  
splAlturacilindro;  
ESCREVER ("O volume do cilindro é ", splVolume);  
FIM.  
[/código]
```

Algoritmos: estruturas de decisão

Até aqui, foi possível notar que os algoritmos se apresentam em estrutura sequencial, o que indica que as instruções eram realizadas de maneira linear, respeitando uma sequência estabelecida, partindo da primeira à última linha. Entretanto é preciso levar em consideração que um programa pode apresentar ou encontrar-se em uma situação na qual seja necessário escolher determinado caminho. Isto indica que as instruções a serem realizadas estarão relacionadas aos resultados gerados por estas situações, segundo afirma William Alves na obra *Linguagem e lógica de programação*, de 2014 (p. 61).

Certamente, você vai verificar que só é viável um programa executar as atividades levando em consideração a condição visualizada ao longo do processamento se existirem estruturas de decisão. É importante lembrar que estas estruturas são as grandes responsáveis pela chamada “inteligência” presente nos programas computacionais.

Iremos observar que as estruturas de decisão mais importantes são classificadas basicamente em SE/ENTÃO, SE/ENTÃO/CASO CONTRÁRIO e FAÇA/CASO. Vale ressaltar que cada linguagem de programação apresenta uma variação no formato ou sintaxe, entretanto, o funcionamento das estruturas é similar.



Estrutura SE/ENTÃO

Usada na análise de uma expressão lógica baseada no seu resultado, o código que adota a cláusula ENTÃO pode ser relacionado ou não. Se, por acaso, o resultado encontrado se apresentar como verdadeiro, isso indica que o código deve ser executado; se for falso, o fluxo do programa se direcionará para a linha que se encontra após o final da estrutura.

Vale evidenciar que é possível realizar uma linha de instrução ou até mesmo um bloco contendo uma variedade de instruções, lembrando que o bloco deve ser limitado pelos comandos INÍCIO/FIM. Segundo o entendimento de William Alves, no livro de 2014 (p. 62), é possível visualizar duas maneiras de uso desta estrutura:

- Através de uma única linha de instrução

SE (expressão lógica) ENTÃO

Instrução;

- Bloco de Instruções

SE (expressão lógica) ENTÃO

INÍCIO

Instrução 1;

Instrução 2;

.

.

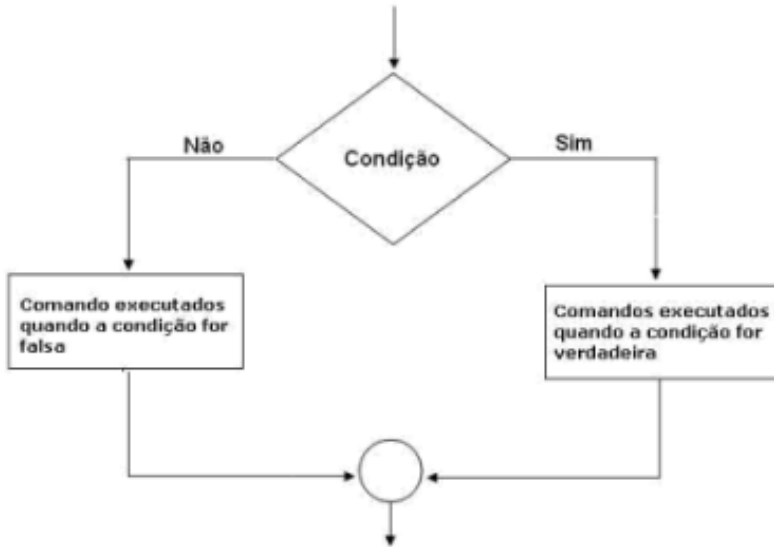
Instrução 3;

FIM

No diagrama a seguir, é possível observar o fluxograma referente a essa estrutura.



Diagrama 4. Diagrama de bloco de estrutura de decisão SE/ENTÃO



Fonte: ALVES, 2014, p. 62. (Adaptado).

[TEXT0]

Condição

Não

Comandos executados quando a condição for falsa

Sim

Comandos executados quando a condição for verdadeira

[/TEXT0]

Observe com atenção: vamos utilizar um exemplo abordado por William Alves em 2014 (p. 62) de um código que exerce a função de estabelecer o nível de temperatura informado pelo usuário. De imediato, é preciso declarar a variável que vamos utilizar no código. Alves (p. 63) informa que o núcleo do código é formado por um conjunto de instruções para apresentar uma mensagem ao usuário e, conseqüentemente, aguardar pela entrada de um valor referente à temperatura medida. Importante frisar que este valor será



arquivado dentro da variável “splTemperatura”, conforme se visualiza na codificação a seguir:

[código]

```
LIMPAR_TELA ( ) ;
```

```
ESCREVER (“digite um valor para a temperatura: ”);
```

```
LER (splTemperatura);
```

[/código]

Certamente, você, na condição de usuário vai observar que, depois dessa fase, vem a chamada “parte inteligente” do programa, que se refere à tomada de decisão com base no comando SE/ENTÃO. Através deste comando, é possível verificar se o valor arquivado dentro da variável (no nosso caso, a splTemperatura) foi maior que 30. Caso esta condição seja confirmada, o usuário irá receber uma mensagem.

[código]

```
SE (splTemperatura>30) ENTÃO
```

```
    ESCREVER (“Essa temperatura é alta! ”);
```

[/código]

A finalização do código ocorre através de uma nova mensagem, pela tecla “Enter” pressionada para confirmar o processamento. É importante deixar claro que este recurso só é acessado através de uma chamada ao comando LER, responsável pela entrada de dados, sem que nenhuma variável seja referenciada. Portanto, após este processamento, é possível verificar como o código desse algoritmo é apresentado. Veja a seguir:



[código]

PROGRAMA Temperatura;

INICIO

VARIÁVEL

SplTemperatura: Simples

LIMPAR_TELA ();

ESCREVER (Digite um valor para a temperatura: ");

LER (splTemperatura);

SE (splTemperatura>30) ENTÃO

ESCREVER ("Essa temperatura é alta! ");

ESCREVER (" Fim do programa ...");

LER ();

FIM.

[/código]

Estrutura SE/ENTÃO/CASO CONTRÁRIO

Vimos como a estrutura SE/ENTÃO é. Executada dentro deste contexto, é possível visualizar uma variação desta estrutura: SE/ENTÃO/CASO CONTRÁRIO. Esta estrutura se caracteriza por apresentar uma cláusula que possibilita a realização de outra instrução ou bloco instrucional, no caso de uma análise não ser verdadeira.

Assim como ocorre na estrutura anterior, para que sejam executadas diversas instruções, é preciso delimitá-las através dos comandos INICIO/FIM. Observe as duas possibilidades:

- Única Linha de instrução

[código]

SE (expressão lógica) ENTÃO

InstruçãoVerdadeira;



CASO CONTRÁRIO

InstruçãoFalsa;

- Bloco de instruções

SE (expressão lógica) ENTÃO

INÍCIO

InstruçãoVerdadeira1;

InstruçãoVerdadeira2;

.

.

InstruçãoVerdadeira3;

FIM

CASO CONTRÁRIO

INÍCIO

Instrução Falsa1;

Instrução Falsa2;

.

.

InstruçãoFalsa3;

FIM

[/código]

Com base no endereço anterior, é possível modificar o código, de maneira que seja apresentada uma mensagem na temperatura normal. A mudança vai se limitar ao comando SE/ENTÃO da seguinte forma:

[código]

SE (splTemparatura>30) ENTÃO

ESCREVER (“Essa temperatura é alta! ”) ;

CASO CONTRÁRIO



```
    ESCRIVER ("Essa temperatura é normal! ");  
[/código]
```

Com isso é possível observar o código completo da seguinte maneira:

```
[código]  
PROGRAMA Temperatura  
INÍCIO  
    VARIÁVEL  
        SplTemperatura: SIMPLES;  
    LIMPAR_TELA ( ):  
        ESCRIVER ("Digite um valor para a temperatura");  
        LER (splTemperatura)  
    SE (splTemperatura>30) ENTÃO  
        ESCRIVER ("Essa temperatura é alta! ");  
    CASO CONTRÁRIO  
        ESCRIVER ("Essa temperatura é normal! ");  
FIM  
[/código]
```

Estruturas SE/ENTÃO Aninhadas

Normalmente, em programação, apresentamos um processo de encadeamento de várias estruturas SE/ENTÃO, o que indica uma estrutura dentro de outra. Isso acontece quando necessitamos analisar outra expressão, levando em consideração o resultado obtido pela INICIO/FIM, o que impede alguma anormalidade quando não



é possível identificar a qual comando a cláusula CASO CONTRÁRIO pertence, como podemos observar no tipo de aplicação a seguir:

[código]

SE (expressão_lógica1) então

Início

SE (EXPRESSÃO_LÓGICA2) ENTÃO

Instrução;

CASO CONTRÁRIO

Instrução;

FIM

CASO CONTRÁRIO

INÍCIO

SE (expressão_lógica3) ENTÃO

Instrução;

CASO CONTRÁRIO

Instrução;

FIM

[/código]

Com base no exemplo anterior, podemos realizar uma análise de Temperatura. É preciso mudar os parâmetros: em uma condição em que a temperatura é maior do que 45, será mostrada uma mensagem “Temperatura muito alta”; abaixo de 30, visualizaremos “Temperatura Alta”, e menor que 10, visualizaremos uma mensagem “Temperatura baixa”, conforme observaremos no código do algoritmo a seguir:

[código]

PROGRAMA Temperatura



INÍCIO

VARIÁVEL

SplTemperatura: SIMPLES;

LIMPAR_TELA ():

ESCREVER (“ digite um valor para a temperatura: ”);

LER (splTemperatura)

SE (splTemperatura>30) ENTÃO

INÍCIO

SE (splTemperatura>45) ENTÃO

ESCREVER (“Temperatura muito alta! ”);

CASO CONTRÁRIO

ESCREVER (“Temperatura é alta! ”)

FIM

CASO CONTRÁRIO

INÍCIO

SE (splTemperatura<10) ENTÃO

ESCREVER (“Temperatura baixa! ”);

CASO CONTRÁRIO

ESCREVER (“Temperatura é normal! ”);

FIM

ESCEREVER (“Fim do programa...”): LER ();

FIM,

[/código]

Estrutura FAÇA CASO

Por fim, temos a estrutura FAÇA CASO, responsável por analisar o conteúdo de uma variável estabelecendo uma comparação com uma diversidade de valores constantes. É essencial frisar que só é



possível realizar uma comparação de igualdade estabelecida entre o valor da variável e as constantes listadas. Desta maneira, não existe a possibilidade de realizar outro modelo de teste. Podemos observar a sintaxe da estrutura a seguir:

[código]

FAÇA CASO (Variável)

Constante1: instrução/bloco de instruções;

Constante 2: instrução/bloco de instruções:

Constante 3: instrução/bloco de instruções;

CASO CONTRÁRIO: instrução/ bloco de instruções:

FIM

:

[/código]



EXPLICANDO

Caso nenhuma das condições seja considerada satisfatória e exista a cláusula CASO CONTRÁRIO, a instrução ou bloco instrucional pertencente a esta cláusula serão realizados de maneira padronizada.

Observe que, na lista de constantes, é possível determinar diversos valores, separando-os através de vírgulas.

Segundo o entendimento de William Alves em *Linguagem e lógica de programação* (p. 68), este modelo de estrutura é bastante utilizado na escolha de alternativas de um menu, como pode ser observar a seguir.



[código]

PROGRAMA Cadastro

INICIO

VARIÁVEL

intOpcaoMenu: INTEGER;

lgRepetir: LÓGICO;

lgRepetir = Verdadeiro

REPITA

LIMPAR_TELA ();

ESCREVER (“ Selecione uma das opções do menu”);

ESCREVER ();

ESCREVER (“1 – Cadastrar cliente”);

ESCREVER (“2 – Alterar dados de cliente”);

ESCREVER (“3 – Excluir registro de cliente”);

ESCREVER (“4 – Imprimir ficha do cliente”);

ESCREVER (“0 – Sair de programa”);

LER (intOpcaoMenu)

FAÇA CASO (intOpcaoMenu)

1: CadastrarCliente:

2. AlterarCliente

3. ExcluirCliente

4. ImprimirCliente

5. lgRepetir:Falso;

CASO CONTRÁRIO: ESCRIVER (“Escolha uma das opções”);

FIM

ATÉ QUE (lgRepetir = Falso)

FIM

[/código]



Esse tipo de código aponta uma lista de alternativas e solicita que seja selecionada qualquer uma delas. Através da estrutura condicional FAÇA CASO, o valor inserido na variável `intOpcaoMenu` é comparado às constantes. É importante ressaltar que um procedimento não visualizado no código é realizado levando em consideração o valor presente na variável.



IMPORTANTE

Observamos, ao longo dessa unidade, que aspectos relacionados ao uso dos algoritmos no contexto da lógica de programação.

Inicialmente, vimos que o aprendizado da lógica de programação é um elemento essencial para os profissionais da área de desenvolvimento, servindo de base para entender a linguagem de programação. Procedimentos ligados aos algoritmos armazenam informações temporariamente, e os dados arquivados longo da execução de um algoritmo são denominados de variáveis.

Vimos conectivos lógicos E, Ou e Não (And, Or e Not), adotados na programação de computadores para estabelecer uma combinação entre duas condições; e as tabelas-verdade, que estabelecem o resultado das combinações realizadas. Tais relações ajudam a guiar o trabalho do programador.

Ao longo desta unidade, com exemplos práticos e ilustrativos, foi possível notar que os algoritmos demonstram uma estrutura sequencial, partindo da primeira até a última linha. Porém, é preciso considerar que um programa pode indicar que as instruções a serem realizadas estarão relacionadas aos resultados gerados por estas situações.

Por fim, abordamos constantes e estruturas de decisão, sempre mostrando como são compostas as respectivas estruturas.



Objetivos

1. Apresentar as características do programa Visualg, evidenciando os seus conceitos e aspectos principais;
2. Abordar as estruturas de repetição, evidenciando as suas funcionalidades dentro de programas Visualg;
3. Observar os principais conceitos referentes aos vetores, verificando os seus comandos e atividades principais dentro do Visualg.



Visualg: fundamentos

Quando nos referimos à ideia de projeção e desenvolvimento de algoritmos, certamente vamos discutir os conceitos e fundamentos que tratam do Visualg. Mas o que significa? Bem, o Visualg pode ser entendido como um programa que tem a função de **testar a lógica no desenvolvimento de um algoritmo**. Por meio desse programa, é possível simular ambientes de programação ao avaliar algoritmos similares ao programa. Tal programa se caracteriza pela disponibilidade gratuita facilmente adquirida na internet em sites de buscas, como o Google e o Yahoo.

A versão mais atualizada do programa é o Visualg 3.0, que é planejado de maneira debugada para os usuários iniciantes no mundo da programação. Por meio desse programa, é possível desenvolver, manipular e até debugar algoritmos empregando uma linguagem de programação similar com a utilizada na língua portuguesa. Diante disso, torna-se mais simples entender a forma de programar adotando o Visualg 3.0 ao invés da forma direta dentro de uma linguagem de programação classificada como comercial; sendo assim, o uso do Visualg 3.0 se direciona a dois aspectos básicos: o aprendizado dos algoritmos e da lógica de programação, com o detalhe muito importante de superar o desafio do inglês. Vale ressaltar que o instalador do Visualg 3.0 está inserido no SourceForge ou no Visualg.



EXPLICANDO

Importante deixar claro que o Visualg 3.0 é considerado, atualmente, um dos programas de computador mais utilizados no mundo, com uma distribuição de mais de 20 milhões de cópias, levando conteúdo a milhares de instituições de ensino.



Quando observamos o menu que compõe o Visualg, visualizamos que ele é composto por algumas partes. Segundo o manual, os principais componentes são subdivididos em:

Arquivo, que se caracteriza por apresentar os comandos responsáveis por abrir, salvar e imprimir algoritmos.

Dentro desta seção, é possível visualizar outros itens como o **“novo”**, que é responsável por desenvolver uma estrutura para o pseudocódigo, alterando o texto inserido no editor. Caso ocorra uma modificação no texto anterior, o Visualg irá solicitar a sua confirmação com o objetivo de salvá-lo antes da sua sobreposição. No meu **“abrir”**, é possível conferir o texto de um pseudocódigo que fora gravado anteriormente, alterando aquele que se encontra no editor.

O menu **“salvar”** tem a função de salvar, de imediato, o texto que se encontra no editor. Caso um novo texto seja gravado pela primeira vez, o Visualg vai solicitar o nome do arquivo, além da sua localização. Dentro desse contexto, o **“salvar como”** possibilita salvar o texto inserido no editor, apresentando uma janela que permite selecionar o nome do arquivo e a sua localização.

Por fim, temos as atividades relacionadas ao **envio por e-mail**, do texto que se encontra no editor: a função **imprimir** os algoritmos correntfcv e expondo anteriormente a janela de configuração de impressão e o menu **“sair”**, que permite ao usuário sair do programa Visualg.

Editar: formado por um conjunto de comandos pertencentes a um editor de texto como o **“copiar, cortar”**, por exemplo. Além desse comando, existem outras opções.

O **corrigir indentação** é responsável pela correção automática da indentação do pseudocódigo. Isso é realizado por meio da tabulação do comando interno com espaços inseridos à esquerda. Outro comando é o **“gravar bloco de texto”**, cuja função é possibilitar a gravação de um texto escolhido em forma de arquivo no editor. Por fim, o comando **“inserir bloco de texto”** possibilita a inserção do conteúdo dentro de um arquivo.



Exibir: caracteriza-se por apresentar os comandos para ativar/desativar alguns seguintes aspectos. Um destes aspectos se refere ao número de linhas responsável por ativar/desativar a apresentação a numeração das linhas na área à esquerda do editor. Importante frisar que a numeração corrente da posição do cursor é apresentada de imediata na barra de status localizada na área inferior presente na tela. Por questões técnicas, a numeração passa por uma desativação ao longo da realização do pseudocódigo. Outro aspecto se refere às variáveis alteradas, que é responsável pela ativação/desativação da apresentação da variável que está sendo alterada. Pensando em uma quantidade extensa de variáveis, é preciso deixar claro que algumas destas podem se encontrar fora da janela de visualização. Você pode notar à medida que esse aspecto se encontra ativado, o Visualg apresenta uma grade de exibição, o que torna a variável visível no instante da alteração.

Importante frisar que estes recursos são essenciais para a realização de um pseudocódigo seguindo todos os procedimentos. Outro ponto relevante é o fato da configuração padronizada deste aspecto se encontrar desativada no momento em que o pseudocódigo está em execução automática. Porém, é preciso clicar no botão para realizar, de maneira automática, a exibição ativada “lembrando que, ao fim desta execução, a configuração retorna à sua fase de desativação.

Pseudocódigo: caracteriza-se por apresentar os comandos que tratam da execução do algoritmo.

Dentro da seção pseudocódigo, é possível verificar alguns comandos como o “executar”, que é o responsável por dar início ou prosseguir na execução do pseudocódigo de maneira automatizada. Outro comando é o passo a passo, que começa ou dá continuidade à execução das linhas do pseudocódigo. Isto permite ao usuário a chance de observar o fluxo de execução, dentre outros aspectos. Outra funcionalidade é o “executar com timer”, que se caracteriza por inserir um atraso anterior à execução da linha, individualmente.

O comando parar permite encerrar imediatamente a realização do pseudocódigo. Certamente, este componente será desabilitado no momento em que não está sendo executado. O comando



“liga/desliga breakpoint” adota e retira um ponto de parada na linha em que o cursor se encontra. Vale ressaltar que estes pontos de parada são essenciais para a depuração e observação da execução dos pseudocódigos, já que possibilitam a análise dos valores pertencentes às variáveis e notam a pilha de ativação pertencente aos subprogramas. Outro aspecto é o comando “**desmarcar todos os breakpoints**”, cuja função é desativar todos os breakpoints que se encontram ativados naquele instante.

Ao inserir o comando “**executar em modo DOS**”, verificaremos como essa alternativa ativada no instante da entrada e da saída padronizada se transforma em uma janela similar ao DOS, que simula a realização de um programa dentro desse ambiente. O comando “**gerar valores aleatórios**” é responsável por ativar a criação de valores aleatórios que modificam a digitação de dados.

Ao final, temos o comando **perfil**, que se caracteriza por ser realizado depois da execução de um pseudocódigo e que demonstra a quantidade de vezes que as linhas foram executadas. Quando se trata da **pilha de ativação**, essa se caracteriza por apresentar uma pilha de subprogramas ativados em certo instante. É adequado usar este comando agrupado em uma série de breakpoints ou com a execução passo a passo.

Exportar: é um aspecto que possibilita a exportação de algoritmo ao realizar uma tradução do Portugol para a linguagem Pascal (PascalZim).

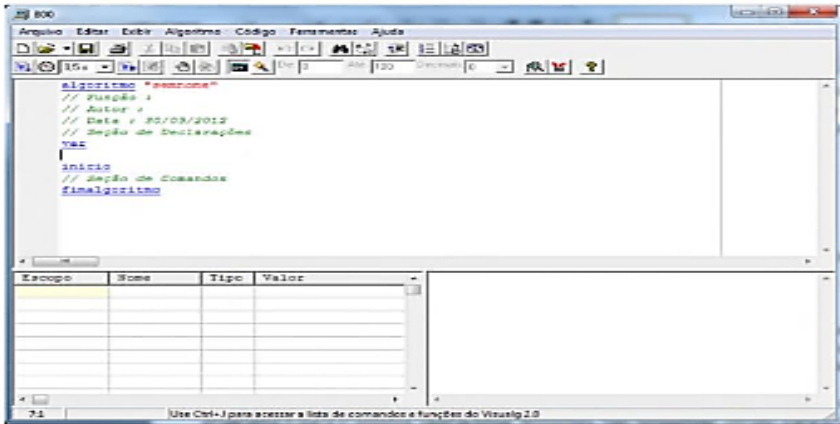
A configuração é um menu em que é permitido configurar alternativas do Visualg, como cores e modelos de letras para pseudocódigo, por exemplo.

Ajuda: permite o acesso às páginas de auxílio e informações referentes ao uso do Visualg.

A sua instalação é bastante simplificada, basicamente confirmando as telas de que vão surgindo ao longo do processo. É importante ressaltar que o Visualg é desenvolvido para o Windows, ou seja, outros sistemas operacionais, como o Linux, não possuem esta versão. Após o download e o procedimento de instalação realizado, é preciso abrir o programa Visualg, conforme se observa na Figura 1:



Figura 1. Tela do Visualg.



Fonte: QUIERELLI, 2012, p. 22.

É possível notar que a tela se subdivide em três setores. O primeiro setor é a parte superior, que se caracteriza por ocupar a área em que os códigos que formam o algoritmo serão redigidos. Na parte inferior esquerda, temos uma subdivisão em quatro colunas em que são apresentadas as variáveis desenvolvidas pelo algoritmo, mostrando detalhes que pertencem a cada variável. Cada coluna pode ser definida conceitualmente de acordo com a Tabela 1:

Tabela 1. Colunas da tela Visualg

Escopo	Determina o modelo da variável (local ou global)
Nome	Insere a nomenclatura da variável
Tipo	Modelo em que a variável arquiva valores
Valor	Apresenta o valor que a variável contém

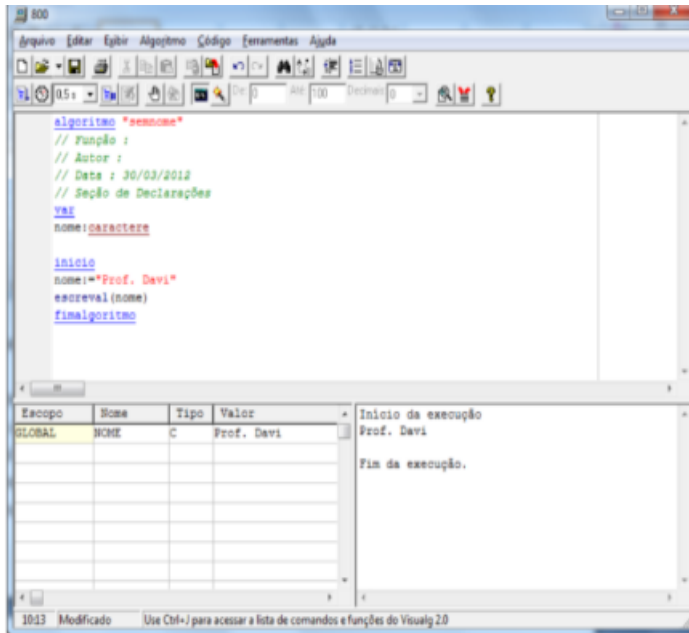
Fonte: QUIERELLI, 2012, p. 22.

Por fim, é possível observar a parte inferior no lado direito da tela em que a execução do algoritmo é de fato realizada, demonstrando o começo, os valores e as mensagens disponibilizadas ao usuário, bem como a finalização do algoritmo. Observando a Figura 2, é possível visualizar a execução do Visualg por meio de uma



variável que classificamos de “nome”, podendo ser visualizada na tela. Você vai perceber que os valores estarão dispostos nos três quadros do Visualg.

Figura 2. Execução do Visualg.



Fonte: QUIERELLI, 2012, p. 23.

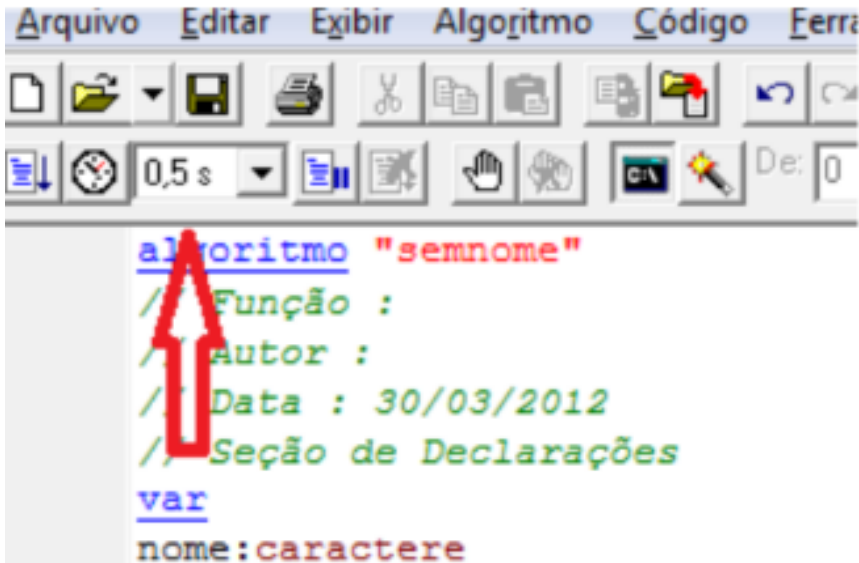
Certamente, um questionamento pode surgir: como executar um algoritmo no Visualg? Uma das maneiras é executar o algoritmo de forma direta sem observar a execução de cada linha individualmente. A outra forma ocorre por meio da utilização de uma técnica para observar a realização de cada linha, em um período determinado para que estas sejam realizadas. O processo direto de execução de um algoritmo precisa necessariamente seguir alguns passos, como ir ao menu “Algoritmo - > Executar” ou até mesmo utilizar um macete (atalho), que é a tecla F9.

Importante frisar que é preciso estabelecer um período preciso em que o passo a passo do algoritmo será realizado nas linhas. Para isso, é preciso escolher dentro da barra de ferramentas



e selecionar o valor almejado para a realização de cada linha, lembrando que o valor de tempo padrão é de meio segundo (0,5s), conforme observamos na Figura 3:

Figura 3. Valor de tempo padrão.



Fonte: QUIERELLI, 2012, p. 23.

Até aqui, observamos como um algoritmo pode ser executado no Visualg; entretanto, é possível notar também como a sua estrutura é desenvolvida para que os algoritmos iniciais possam ser criados. À medida que um Visualg é aberto ou algum novo algoritmo é solicitado por meio do menu “Arquivo -> Novo”, alguns códigos predeterminados são apresentados, demonstrando como funciona a estrutura básica de um algoritmo dentro do Visualg. Na codificação a seguir, trago um exemplo de como funciona na prática esta estruturação:

```
Algoritmo “comnome”  
// Função:  
// Autor:  
// Data: 29/12/2019  
// Seção de Declarações
```



Var
Início
//Seção de Comandos
Fimalgoritmo

Normalmente, um algoritmo começa com um nome determinado, sendo que esse nome se refere à função do algoritmo. No exemplo dado, um algoritmo que apresenta uma nomenclatura foi classificado de “comnome”. Outro aspecto importante é o fato de que as linhas que se encontram abaixo, contendo duas barras anteriores aos códigos, local onde o programador consegue colocar a funcionalidade do código para que algum usuário veja. Importante frisar que estes comentários no código não serão explanados no momento em que o algoritmo for realizado. Em seguida, vem a inserção do “var”, que aponta o local onde ocorrem as declarações de variáveis para que sejam utilizadas no corpo de algoritmo.

Tabela 2. Termos utilizados no Visualg

Algoritmo	Palavra usada para nomear o programa.
Var	Informa o começo da sessão para que as variáveis e constantes sejam declaradas.
Início	Aponta o início da sessão de comandos pertencentes ao programa.
Fimalgoritmo	Apresenta o valor que a variável contém.

Fonte: PASCHOALINI, 2017, p. 60.

É preciso apresentar um nome para cada variável e, em seguida, estabelecer o modelo dela separado pelo caractere “:”.

Exemplo:

Var
Nome: caractere
Valor, total: real.



EXPLICANDO

Quando são disponibilizadas variáveis do mesmo modelo, é preciso separá-las por uma vírgula; caso o modelo seja diferente, a variável será colocada em outra linha.

Logo após a seção de variáveis, aponta o corpo do algoritmo limitado por meio dos comandos “início/finalgoritmo”. Entre esses comandos serão inseridos outros com a lógica para o desenvolvimento de um algoritmo que alcance o objetivo.

Estruturas de repetição

Quando tratamos do conceito de linguagem de programação, é preciso compreender que toda linguagem, inclusive a Assembly, apresenta uma variedade de comandos que possibilita a realização repetitiva de um trecho delimitado de código pertencente a um programa. Com esse recurso, o programador não terá a necessidade de inserir uma duplicação nas redes de código, pois será preciso executá-las em por diversas vezes. A variedade de comandos apresentada é classificada como estruturas de repetição, também conhecidas como **loops (laços)**.

A linguagem Assembly adota comandos de compreensão mais simples pelos programadores em relação à linguagem de máquina. Vale frisar que cada instrução da linguagem de máquina apresenta um comando similar à linguagem Assembly. Esse tipo de linguagem normalmente é mais veloz.

Importante ressaltar que as variações mais antigas referentes à linguagem BASIC apresentavam somente a estrutura FOR/NEXT; entretanto, as linguagens mais atuais apresentam mais alternativas



(a Visual Basic é um exemplo). Ao longo desta seção, observaremos as estruturas mais importantes, lembrando que essas são denominadas também de **laços de repetição**.

Laço com repetição predefinida

Para tratarmos de um laço com repetição predefinida, é preciso, de imediato, conceituá-la. A estrutura definida como REPITA PARA/DE ATÉ se caracteriza por realizar um trecho de código numa quantidade invariável de vezes definida pelo programador. É o modelo de estrutura mais simplista a ser executado e será usado somente quando se conhece, de forma antecipada, a quantidade de vezes que a linha de código deverá ser realizada.

Vale ressaltar que uma variável estabelece um controle da quantidade de vezes que o código foi de fato executado. Ao alcançar a limitação adotada pelo programa, há um encerramento da repetição e ele continua a ser executado na linha posterior à estrutura, conforme a sintaxe a seguir:

REPITA PARA NomeVariável DE início ATÉ fim

Note que a terminologia “início” simboliza o valor inicial a ser inserido na variável de controle. O termo “fim” estabelece o valor final que restringe as execuções do laço. No momento que o código é executado inicialmente, a variável de controle se iguala ao valor inicial. Ao longo da execução realizada no laço, automaticamente nós vamos notar que esse valor passa a ser incrementado. Havendo igualdade nos valores em relação ao limite, haverá um encerramento do laço.

Para que fique claro o entendimento, vamos a um exemplo prático: imagine um algoritmo que possibilita a criação de uma tabuada de números de multiplicação entre 11 e 20, inicialmente



partindo de um número disponibilizado pelo usuário. De imediato, é fornecida uma descrição das operações a serem realizadas para, posteriormente, apresentarmos o código do algoritmo redigido na pseudolinguagem.

Tabela 3. Descrição narrativa

1. Solicitar a informação do número relacionado à tabuada, pelo usuário.
2. Dar início ao contador de repetições.
3. Realizar o cálculo da multiplicação do contador por meio do número disponibilizado pelo usuário.
4. Apresentar o valor referente à multiplicação.
5. Inserir o incremento no contador de repetições.
6. Observar se o nível do contador não é maior que 20. Caso não seja preciso, retornar ao procedimento número 3. Caso seja, a execução deverá ser consultada.

Código em pseudolinguagem

PROGRAMA TabuadaMultiplicacao;

INÍCIO

VARIÁVEL

intFator, intProduto, intNumero: INTEIRO;

LIMPAR_TELA () ;

ESCREVER ("Digite o número desejado: ");

LER (intNumero);

REPITA PARA intFator DE 11 ATÉ 20

ESCREVER (TEXTO (intNumero) + "X" + INT_PARA_TEXTO
(intFator) + "=" +INT_PARA_TEXTO (intNumero * intFator));

FIM



Observando o código inserido, é possível observar que, nesse novo algoritmo, outras funções novas são utilizadas: a função `LIMPAR_TELA ( )`, que tem como finalidade extinguir todo e qualquer texto redigido no monitor; já a função `INT_PARA_TEXTO( )` é responsável por transformar um valor numérico em um conjunto de caracteres relacionados. Importante lembrar que as duas funcionalidades compõem as bibliotecas de procedimentos diversas funções presentes nas linguagens de programação.

Um aspecto importante deve ser analisado: o código normalmente solicita ao usuário que o mesmo escreva um valor a ser usado no laço para criar a tabuada. Você irá notar que a linha de código reproduzida é constituída pelo comando `ESCREVER ( )`, que abrange uma série de caracteres que apresentam o valor escrito pelo usuário, assim como o fator multiplicador, culminando no resultado da multiplicação destes valores. A execução de uma variedade de linhas de código depende da sua inserção em blocos, cujas delimitações são norteadas pelos comandos `INÍCIO` e `FIM`. Na ocorrência de uma linha, dispensam-se esses comandos.

A estrutura `REPITA PARA/DE ATÉ` apresenta uma cláusula não obrigatória denominada de `INCREMENTO`. E como isso ocorre? Bem, geralmente há um incremento na variável de controle dentro de uma unidade a cada interação que surge, sendo que seu valor passa por uma atualização ao se somar com o número 1. Com essa cláusula, é possível modificar o valor de incremento, conforme se observa na codificação a seguir:

```
PROGRAMA NumerosPares;
INÍCIO
    VARIÁVEL
        intNumero: INTEIRO;

        LIMPAR_TELA ( ) ;
        ESCREVER ("Números pares até 250");

        REPITA PARA intNumero DE 0 ATÉ 250 INCREMENTO 2
            ESCREVER (intNumero);
FIM.
```



Outro ponto importante é fato de que, se determinarmos um valor negativo, haverá uma redução do valor da variável, como no exemplo a seguir:

```
PROGRAMA ContagemRegressiva:  
INÍCIO
```

```
VARIÁVEL
```

```
    intNumero: INTEIRO;
```

```
    LIMPAR_TELA ( ) ;
```

```
    ESCREVER ("Contagem Regressiva iniciando em 200");
```

```
    REPITA PARA intNumero DE 200 ATÉ o INCREMENTO -1
```

```
        ESCREVER (intNumero);
```

```
FIM.
```

Repetição com teste no início

Quando nos referimos à estrutura de repetição **FAÇA ENQUANTO**, é preciso compreender que a mesma deve ser usada no momento em que não se sabe, de forma antecipada, o número de vezes que o código deverá ser de fato executado. Só pra se ter uma ideia, o fator que determina o encerramento de uma execução está ligado à ação do usuário, como uma condição encontrada no programa, por exemplo.

Ao invés da variável de controle que se caracteriza por apresentar a quantidade de vezes que o código foi reproduzido, a estrutura **FAÇA ENQUANTO** adota uma expressão lógica que normalmente volta a um valor, seja verdadeiro ou falso, depois das interações. No momento em que o resultado for considerado verdadeiro, o código será realizado. A estrutura **FAÇA ENQUANTO** avalia de imediato a



expressão no período anterior à execução do código ou de suas linhas. Se porventura retornar um valor falso de imediato nas primeiras avaliações, o código não será realizado nenhuma vez.

É possível falar sobre estas estruturas de repetição em que o teste é realizado no início por meio do exemplo do caixa eletrônico. Certamente, seria muito estranho se durante as operações bancárias esses terminais não exigissem o uso de senhas. Diante disso, é possível afirmar então que esse sistema consegue implementar um processo de repetição com teste no início, ou seja, o sistema analisa a senha digitada antes da operação ser executada e, logo em seguida, questiona se desejamos continuar com a operação. Isso indica que o sistema não vai realizar nenhum procedimento de repetição sem testar uma condição antes. Enquanto uma condição for verificada, as ações desejadas serão repetidas.

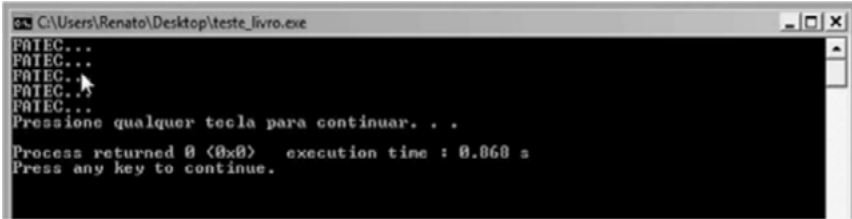
Alguns conceitos devem ser levados em consideração no momento em que se utilizam estas estruturas de repetição. Eles podem ser definidos como:

- **Contador:** é responsável por controlar a quantidade de repetições no momento em que se sabe quantas foram definidas. Para evidenciar esse conceito de maneira mais clara, veja o exemplo, que utiliza a expressão “FATEC” cinco vezes na saída padrão (tela):

```
#include< stdio.h>
#include < stdlib.h>
main( )
{
    int contador=0; // aqui criamos o contador de repetições com valor
    inicial de 0 while (contador < 5) // enquanto a contagem for menor
    que 5 (ou seja, de 0 a 4)
    {
        printf(“FEDERAL... \n”);
        contador++; // o contador incrementa de valor, a fim de passar-
        mos adiante
    }
    system(“PAUSE”);
}
```



Figura 4. Exemplo de contador.



Fonte: SOFNER, 2013, p. 65.

- **Incremento (ou decremento):** responsável por reduzir ou aumentar o valor atribuído ao contador, visando alcançar o número de repetições desejado. A intenção é evitar o chamado loop infinito (repetição eterna), o que criaria um resultado indesejado pelo programa. Para evidenciar esse conceito de maneira mais clara, veja o exemplo a seguir com um programa que apresenta contagem regressiva de 10 a 0:

```
#include< stdio.h>
#include < stdlib.h>
main( )
{
    int cont=10;
    while (cont >= 0)
    {
        printf("%d \n", cont);
        cont--;
    }
    printf("We have a liftoff! \n");
    system("PAUSE");
}
```



Figura 5. Exemplo de Incremento.

```
C:\Users\Renato\Desktop\teste_livro.exe
10
9
8
7
6
5
4
3
2
1
0
We have a liftoff!
Pressione qualquer tecla para continuar. . .
Process returned 0 (0x0) execution time : 3.717 s
Press any key to continue.
-
```

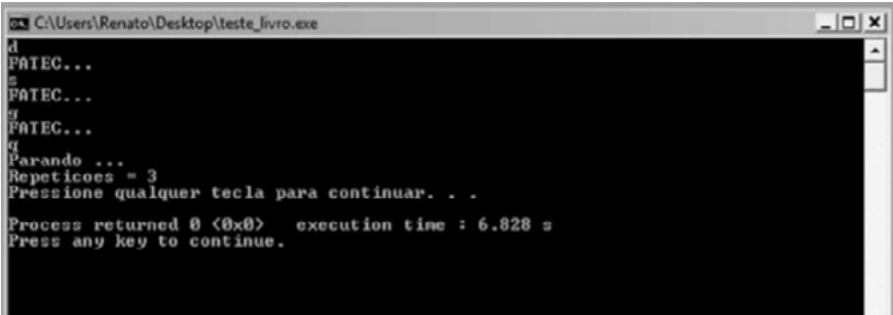
Fonte: SOFNER, 2013, p. 66.

- **Acumulador:** responsável pela inserção de entradas de dados de cada iteração de repetição. Isso cria uma soma que será utilizada no momento da saída da repetição. Para evidenciar esse conceito de maneira mais clara, veja o exemplo a seguir:

```
#include<stdio.h>
#include <stdlib.h>
main( )
{
    int cont=0; // contador inicializado com valor zero – sempre recomendável
    char letra;
    while (letra=getchar() != 'q') // função getchar( ) captura o caractere digitado
    {
        printf("FATEC... \n");
        fflush(stdin);
        cont++; // incrementando o contador
    }
    printf("Parando ... \n");
    printf("Repeticoes = %d \n", cont); // mostrando o número de repetições
    efetuadas system("PAUSE");
}
```




Figura 6. Exemplo de acumulador.



```
C:\Users\Renato\Desktop\teste_livro.exe
d
FATEC...
s
FATEC...
q
FATEC...
q
Parando ...
Repeticoes = 3
Pressione qualquer tecla para continuar. . .
Process returned 0 (0x0)   execution time : 6.828 s
Press any key to continue.
```

Fonte: SOFNER, 2013, p. 67.

- **Critério de parada:** indica a condição para finalizar uma repetição quando não se sabe quantas estão sendo realizadas. Para evidenciar esse conceito de maneira mais clara, veja o exemplo de um programa de repetição que indica uma condição de saída com caractere e acumula os valores inseridos:

```
#include< stdio.h>
#include < stdlib.h>
main()
{
    int x; int cont=0; int total=0;
    char letra;
    while (letra=getchar() != 'q')
    {
        printf("Digite um numero: \n");
        scanf("%d", &x);
        fflush(stdin);
        cont++;
        total = total + x;
    }
    printf("Parando ... \n");
    printf("Repeticoes = %d e total = %d. \n", cont, total);
    system("PAUSE");
}
```



Repetição com teste no fim

Por fim, as estruturas de repetição vão tratar de REPITA/ATÉ QUE. Ela se distingue da estrutura FAÇA ENQUANTO pelo fato de realizar a análise da condição no final do laço. Dessa maneira, o código sempre será realizado no mínimo uma vez. Esse tipo de estrutura também é utilizado no momento em que se desconhece o número de vezes que o código deve ser realizado, assim como ocorreu no laço anterior o fim da repetição é estabelecido por meio de uma condição.

Visualg: estrutura de repetição

Diante do que vimos, é de grande importância a estrutura de repetição para o desenvolvimento de algoritmos. Nesse contexto, o programa Visualg emprega essa estrutura disponibilizando aquelas que são mais utilizadas na linguagem de programação. São eles: o laço contado para...até...faça, que funciona de maneira parecida ao for...to...do pertencente à linguagem Pascal; o laço condicionado enquanto...faça, que funciona similarmente ao while...do; e, por fim, a estrutura repita...até, que é parecida com a repeat...until. Veremos como funciona a sintaxe desses comandos.

Para...faça

Esse modelo de estrutura se caracteriza por repetir uma sequência de comandos com uma quantidade de vezes estabelecida. Este comando pode ser redigido como no exemplo a seguir:

Para <variável> de <valor-inicial> até <valor-limite> [passo <incremento>] faça

 <sequencia – de – comandos>

Fim para

Para ficar mais claro, é preciso que você conheça cada um dos termos que compõem este comando (estrutura). Uma variável pode ser definida como a responsável pelo controle da quantidade de repetições do laço. O valor inicial é visto como uma expressão que



determina o valor da inicialização da variável no período anterior a primeira repetição atribuída no laço, diferente do valor limite, que determina o valor máximo alcançado pela variável contadora.

O incremento, que também faz parte da composição deste comando, é visto como um item opcional, pois quando se encontra presente, acompanhado da palavra “passo”, é definido como uma expressão que determina o incremento que será inserido à variável contadora toda vez que o laço for repetido. No momento em que essa expressão não é de fato empregada, o valor padrão incluído ao incremento é definido como um. Outro aspecto que devemos nos atentar é o fato de que existe a possibilidade de determinar valores negativos para o incremento, porém, se o valor encontrado for nulo após a avaliação da expressão, a execução do algoritmo normalmente é interrompida, sendo exibida uma mensagem de erro.

Ao final, o termo “fimpara” aponta o final da sequência de comandos que serão repetidos. Importante frisar que, toda vez que o programa atingir este ponto, o valor do incremento será inserido à variável contadora e será comparado ao valor limite. No caso do valor se apresentar menor ou igual (no caso de o incremento exibir valores negativos), haverá uma diversidade de execução das sequências de comando; se ocorrer o contrário, a execução ocorrerá a partir do primeiro comando depois do termo “fimpara”.



EXPLICANDO

Os termos utilizados referentes ao <valor-inicial>, <valor-limite> e <incremento> são analisados uma única vez no período anterior a primeira repetição e não mudam ao longo da execução do laço, mesmo que algumas variáveis de maneira eventual apresentem valores alterados.

Observe o exemplo a seguir e visualize os números de 11 a 20, que são expostos em ordem crescente.



```
algoritmo "Números de 1 a 10"  
var j: inteiro  
inicio  
para j de 1 ate 10 faca  
    escreva (j:3)  
fimpara  
fimalgoritmo
```

Agora, se a primeira repetição <valor- inicial> for maior que <valor-limite> ou até mesmo quando o <incremento> apresentar valores negativos, não haverá execução dos laços.

```
algoritmo "Números de 10 a 1 ( não funciona)"  
var j: inteiro  
inicio  
para j de 10 ate 1 faca  
    escreva (j:3)  
fimpara  
fimalgoritmo
```

Inserir vídeo - link <https://youtu.be/WDpH96xEoT4> (0: 23' - 3:25')

Enquanto...faça

Esse modelo de estrutura se caracteriza por repetir uma sequência de comandos no momento em que uma condição, normalmente determinada por uma expressão lógica, for atendida. Esse comando pode ser redigido como no exemplo a seguir:

```
enquanto <expressão-lógica> faca  
    <sequência-de-comandos>  
fimenquanto
```

Para ficar mais claro, é preciso que você conheça cada um dos termos que compõem esse comando (estrutura). Uma expressão lógica se caracteriza por ser analisada no momento anterior a cada repetição do laço. Importante ressaltar que, quando o resultado



exposto é verdadeiro, a <sequência - de - comando> será realizada. Já o “fimenquanto” aponta o final da <sequência-de-comando> que irá se repetir. Toda vez que a execução consegue atingir este ponto, é preciso retornar ao começo do laço, fazendo com que a <expressão - lógica> seja avaliada. Caso o resultado desta análise for considerado como verdadeiro, a < sequência - de - comandos> será realizada mais uma vez; se ocorrer o contrário, a execução vai prosseguir depois do primeiro comando após o fimenquanto.

Observe o exemplo a seguir e visualizaremos como é possível resolver o problema por meio desta estrutura de repetição:

```
algoritmo “Números de 1 a 10 (com enquanto...faca)”  
var j: inteiro  
inicio  
j <- 1  
enquanto j <= 10 faca  
    escreva (j:3)  
    j <- j + 1  
fimenquanto  
fimalgoritmo
```

Vale ressaltar que o laço enquanto...faça avalia sua condição de parada no momento anterior à execução da sua sequência de comandos, que pode ser executada zero ou por diversas vezes.

Repita...até

Por fim, esse modelo de estrutura se caracteriza por repetir uma sequência de comandos no momento em que uma condição, normalmente determinada por uma expressão lógica, for atendida. Esse comando pode ser redigido como no exemplo a seguir:

```
repita  
    <sequência-de-comandos>  
ate <expressão-lógica>
```

Para ficar mais claro, é preciso que você conheça cada um dos termos que compõem este comando (estrutura). O termo “repita”



se caracteriza por indicar a fase inicial do laço. Já o termo “até <expressão - lógica>” aponta o final da <sequência-de-comandos> a ser reproduzida. Toda vez que o programa atingir este ponto, <expressão-lógica> é analisada: caso o resultado seja considerado como FALSO, serão executados os comandos que se encontram entre as linhas repetidas: em uma situação contrária à execução, irá prosseguir partindo do primeiro comando depois desta linha.

Observe o exemplo a seguir e visualize como é possível resolver o problema por meio desta estrutura de repetição:

```
algoritmo “Números de 1 a 10 (com repita)”  
var j: inteiro  
início  
j <- 1  
repita  
    escreva (j:3)  
    j <- j + 1  
ate j > 10  
finalgoritmo
```

Vale ressaltar que o laço **repita...até** avalia sua condição de parada no momento posterior à execução da sua sequência de comandos, que pode ser executada uma ou por diversas vezes.



Vetores e matrizes

Para compreendermos a funcionalidade dos vetores, é preciso ter em mente que estamos tratando da ideia referente à estrutura de dados. É essencial considerarmos que a manutenção e preservação dos dados devem ocorrer em dispositivos de arquivamento. Neste contexto, iremos tratar das “variáveis indexadas” que arquivam valores de um mesmo modelo e são essenciais para a manipulação de diversos valores similares uni ou multi dimensionados.

Conceitualmente, um vetor é definido como uma estrutura que arquiva diversos dados do mesmo tipo, diferente das variáveis de forma individualizada. Quando se trata de programação, podemos considerá-la como uma das estruturas mais simples. Importante frisar que os elementos individuais são visualizados graças a sua posição em um vetor. A posição é disponibilizada por meio de um índice que adota uma sequência de números inteiros ofertados de maneira eficaz. Vale frisar que o vetor é declarado da seguinte maneira:

`Tipo_dos_elementos    nome_de_vetor    [número    de elementos]`

Os elementos do vetor têm o índice como referência e se iniciam no zero. Sendo assim, o vetor apresentará o valor do componente inicial da estrutura. Esses elementos geralmente são visualizados ao se percorrer a estrutura por uma rotina similar a este que visualizamos a seguir:

```
Void percorrer_vetor( )
{
  Int i;
  For(i=0; i<10; i++)
  {
    s[i] = , // vamos aqui preencher as posições do vetor com o número 8
  }
}
```



Um aspecto importante que devemos frisar é o fato de que sempre que for preciso declarar um vetor, o seu espaço é inserido de maneira contínua dentro da memória. A posição ocupada na memória é adquirida por meio do nome do vetor, que apresenta o endereço que consta no primeiro componente.

Ao passarmos um endereço, iremos notar que a variável que o adquire funcionará como uma espécie de ponteiro direcionado aos modelos do vetor. Sendo assim, no início de uma função (cabeçalho) – em que o vetor é recebido na condição de argumento – surge, geralmente, um ponteiro adquirindo um parâmetro respectivo.

Se for necessário dar início a este vetor utilizando um valor nos elementos, é possível transmiti-los na condição de um argumento de uma função, com a missão de executar uma tarefa específica.

```
#include <stdio.h>
#include <stdlib.h>

Void inicializa (int s {10})
{
    Int i;
    For(i=0; i<10; i++)
    {
        S[i]=0; // vamos preencher as posições do vetor com “0”
    }
}

Void mostra (int s[10])
{
    Int i;
    Printf (“ O vetor ficou assim: \n”); // vamos mostrar as posições do vetor
    For(i=; i<10; i++)
    g{
        Printf(“| %d “,s[i]);
    }
    Printf(“|”);
    Printf(“\n\n”);
}

Main( )
```




```
{  
  
    Int v{10};  
    Inicializa(v)  
    Mostra(v)
```

```
System("pause");  
}
```

Um aspecto que devemos tratar aqui se refere às strings. Nós vamos perceber que a linguagem C normalmente não estabelece um modelo específico para manipular as strings, que são vetores cuja terminação apresenta o caractere null. Sendo assim, ela disponibiliza uma biblioteca de funções mais específicas, como a “string h”, por exemplo.

As funções que tratam da manipulação das strings fazem um percurso até visualizar o caractere null, quando ficarão sabendo do seu encerramento. É possível empregar, por exemplo, um caractere zero presente na tabela ASCII com o objetivo de encerrar a string. Vale lembrar que este caractere abrange um espaço previsto pelo programador, o que nos dá uma ideia de que a relação existente entre strings e vetores vai ocorrer de maneira direta. É possível considerar que uma string é um vetor de caracteres; entretanto, nem todo vetor pode ser considerado uma string.

Vamos imaginar uma declaração e inicialização de uma string: uma string pode arquivar cerca de 20 caracteres, levando em consideração o null.

```
Char frase [20] = "Teste"; // usando 5 + 1 posições  
Eis um programa que pode ilustrar essa questão:
```

```
#include <stdio.h>  
#include <stdlib.h>  
Main ( )  
{  
  
    Char frase [50];  
    Int i;  
    For(i=0; i<.30;i++)  
    {
```



```
Frase[i] = 'A' + I; /*variável 'i' incrementa a posição do
caractere na Tabela ASCII */
}
Frase[i] = NULL
Printf("A string contem %s \n", frase);

System ("pause");
}
```



EXPLICANDO

Quando se trata de caracteres string, é necessário visualizar as suas diferenças básicas. Um caractere "A" significa um caractere simples, enquanto que um caractere "A" aponta uma cadeia de caracteres que denominamos de strings.

Quando tratamos do tema linguagem de programação, o termo vetor, que também pode ser denominado de array, é conhecido como um modelo especial de variável que normalmente arquiva em si mesmo diversos valores. A acessibilidade a esses valores arquivados nas variáveis é realizada por meio dos seus índices.

De maneira analógica, é possível considerar que a variável - vetor é o ônibus, assim como os assentos serão os índices e os passageiros representando os valores. Com isso, é possível identificar o valor em cada índice dentro da variável - vetor.

Baseado neste exemplo do ônibus, observe o vetor a seguir:

Tabela 4. Vetor: EMPRESA

RECURSOS HUMANOS	ESTRATÉGICO	TÁCITO	OPERACIONAL
1	2	3	4

Ao observar a Tabela 4, é possível identificar algumas características importantes. A primeira delas se refere ao nome atribuído ao vetor, que, no exemplo, foi EMPRESA. No Visualg são utilizadas



as mesmas regras para atribuir os nomes. O segundo aspecto é a dimensão, demonstrando que a variável tem capacidade de arquivar um conjunto de quatro valores que conseguem ser visualizados por meio de índices enumerados de um a quatro. O tipo de dados que armazena é classificado como caractere, já que estamos guardando nomes; entretanto, os vetores podem ser classificados como **inteiro, real ou lógico**.

Se retornarmos ao exemplo do ônibus e relacionarmos como o exemplo exposto, é possível descobrir qual o valor arquivado no índice quatro do vetor empresa que, no nosso caso, trata-se do setor operacional. A partir daí, é possível entender o conceito de vetor e observar a sintaxe de declaração presente na variável vetor, conforme se observa a seguir:

Nome – do – vetor: vetor[índice – inicial..índice-final] de tipo

No exemplo abordado a sintaxe pode ser escrita desta maneira:

EMPRESA: vetor[1..4] de caractere

Esta declaração nos possibilita criar o vetor EMPRESA, que contém quatro espaços com capacidade de armazenamento dos valores do tipo caractere. Para preenchermos o vetor, foi introduzido um comando de entrada, no exemplo, “LEIA”, porém, atentando-se ao fato de saber em qual índice será arquivado o dado e qual o nome do vetor. Seguindo esta mesma linha de raciocínio, é possível escrever o valor arquivado:

ESCREVA (“Digite um nome: ”)

Leia (EMPRESA[1])

ESCREVA (“O nome armazenado no 1º índice é”, EMPRESA[1])

Importante observar que, especificamente nesse caso, o comando LEIA arquivará o nome digitado dentro do vetor EMPRESA, com o índice um. Em seguida, o comando “ESCREVA” apresenta que



o foi arquivado no índice determinado. Agora imagine a situação em que um vetor apresenta uma grande extensão, com mais de 100 índices, por exemplo. Naturalmente, o tempo de digitação destas linhas será longo e os códigos extremamente extensos. O que fazer em uma situação dessas? Bem, se observarmos com atenção, nós vamos reparar que na digitação ou na apresentação de dados nos índices do vetor, nós utilizamos os mesmos comandos, sempre alterando somente o número do índice.

É preciso compreender que estamos tratando de uma atividade repetitiva, tanto que utilizamos um laço de repetição. Dentro desse contexto, o laço ideal que podemos considerar é PARA...FAÇA, pois nos dá a condição de saber a quantidade de vezes que a repetição vai acontecer” com base nos índices inicial e final pertencentes a um valor. Vamos observar mais um exemplo a seguir em que apresentamos a utilização de um vetor para arquivar e disponibilizar nomes. Neste exemplo, vamos exibir o nome dos quatro setores da empresa por meio dos laços de repetição:

Algoritmo: “Exemplo_Vetor”

// Função: Arquivar e apresentar dados presentes no vetor

// Autor: José Fernando Teixeira

// Data: 02/01/2019

// Seção de Declarações

Var

Família: vetor [1...4] de caractere

i: Inteiro

Início

//Seção de Comandos

//Alimentando o Vetor

Para i de 1 até 4 faça

 Escreva(“Nome para o “i,”º índice do vetor: “)

Fimpara

Escreva1

//Exibindo o vetor preenchido

Para i de 1 até 4 faça

 Escreva (família [i], “ “)

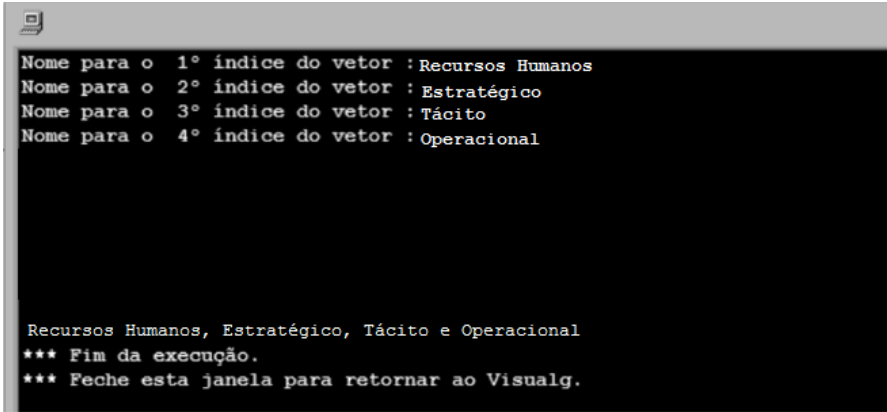
Fimpara

Fimalgoritmo



Representando esta sintaxe em um programa computacional observaremos que o vetor será alimentado e exibido posteriormente da forma como observamos a seguir:

Figura 7. Vetor alimentado e exibido posteriormente.



```
Nome para o 1º índice do vetor :Recursos Humanos
Nome para o 2º índice do vetor :Estratégico
Nome para o 3º índice do vetor :Tácito
Nome para o 4º índice do vetor :Operacional

Recursos Humanos, Estratégico, Tácito e Operacional
*** Fim da execução.
*** Feche esta janela para retornar ao Visualg.
```



IMPORTANTE

Observamos que a ideia de projeção e desenvolvimento de algoritmos está ligada aos fundamentos do Visualg, que é um programa que tem a função de testar a lógica no desenvolvimento de um algoritmo. Por meio dele, visualizamos a simulação de ambientes de programação, observando a sua disponibilidade gratuita facilmente adquirida na internet.

Notamos que toda linguagem de programação apresenta uma série de comandos que permitem a execução repetitiva de um trecho delimitado de código que pertence a um programa. Por meio desse recurso, o programador vai precisar inserir uma duplicação nas redes de código, pois será preciso executá-las por diversas vezes.

Outro aspecto importante que foi notado é o fato de que, para compreendermos a funcionalidade dos vetores, faz-se necessário



compreender o funcionamento das estruturas de dados. Diante desse contexto, é primordial a manutenção e preservação dos dados, que precisam ocorrer dentro de dispositivos de armazenamento. Nesse cenário, tratou-se das variáveis indexadas que armazenam valores de um mesmo modelo e são fundamentais para a manipulação de vários valores similares.



Objetivos

1. Apresentar as características das matrizes evidenciando seus conceitos e aspectos principais;
2. Abordar a aplicação das matrizes no programa VisualG evidenciando suas funcionalidades;
3. Observar os principais conceitos da modularização de algoritmos verificando os seus comandos e atividades essenciais;
4. Analisar o uso da modularização no VisualG observando exemplos e aplicações.



Matrizes

Matriz pode ser definida como uma técnica de programação que permite a execução de atividades de agrupamento de diversas informações dentro da mesma variável. Esse tipo de agrupamento ocorre sempre em obediência a dados semelhantes, de modo que seja nomeado também como estrutura de dados homogênea.

Esse modelo de estrutura possui tipos de dados que podem ser desenvolvidos de acordo com sua necessidade, pois não são todas as vezes que variáveis, mesmo mais básicas, serão suficientes para fazer a representação utilizada por um sistema.

Ao contrário dos vetores, as matrizes podem ter duas ou mais dimensões. Porém, você verá que é comum empregar a nomenclatura “matrizes unidimensionais” para definir os vetores. As matrizes, em geral, servem como um viés facilitador em uma situação a qual teremos muitas variáveis com algoritmos semelhantes e precisamos organizar as informações dispostas, que serão distribuídas entre linhas e colunas.

Veja na Tabela 1 como podemos utilizar uma matriz:

Tabela 1. Exemplo de matriz.

Aluno	Nota 1	Nota 2	Nota 3	Nota 4	Média
1	6.0	7.0	5.0	8.0	6.5
2	8.0	9.0	6.0	9.0	8.0
3	6.0	7.0	8.0	5.0	6.5

Com os valores já dispostos nessa tabela, para efetuar o cálculo das médias de cada aluno, somente é necessário redigir um programa para fazer os cálculos das médias. De acordo com Manzano (2000, p. 68), representando o aluno 1 da tabela, será utilizada a variável MD1, para o aluno 2, MD2, e para o aluno 3, MD3. Caso houvesse mais alunos, seguiríamos essa mesma lógica. No caso acima, temos essa representação:

[Código]

MD1 = 6.5

MD2 = 8.0



MD3 = 6.5

[/Código]

Ainda conforme Manzano (2000), nesse caso deve ser desenvolvido um programa que faça a leitura das notas, uma por uma, a somatória de todas elas e a divisão do valor da soma, por 3, permitindo o resultado da média. Como podemos ver no exemplo da seguinte estrutura:

Código de sintaxe básica da matriz:

[Código]

```
algoritmo "Media_Turma"
```

```
var
```

```
MD1, MD2, MD3: real
```

```
SOMA, MEDIA: real
```

```
inicio
```

```
SOMA <- 0
```

```
leia(MD1, MD2, MD3)
```

```
SOMA <- MD1 + MD2 + MD3
```

```
MEDIA <- SOMA / 3
```

```
escreva(MEDIA)
```

```
finalgoritmo
```

[/Código]

Para obter o número da média, foram usadas três variáveis. Já com a técnica de matrizes é possível utilizar uma variável, armazenando os três valores.

Inicializando matrizes

A inicialização de uma matriz deve ser feita no momento em que ela é criada. Nesse processo será preciso atribuir um valor inicial e final, assim como ocorre com os vetores. O trecho de código a seguir exemplifica a inicialização do vetor teste.

[Código]

```
int identidade[6][6]={1,0,0,0},{0,1,0,0},{0,0,1,0}, {0,0,0,1} };
```

[/Código]



Matrizes bidimensionais

As matrizes bidimensionais possuem duas ou mais linhas e colunas e vetores que representam uma estrutura homogênea. Em uma declaração de matriz, algumas informações são primordiais para a sua criação, a exemplo da quantidade de linhas, colunas e o seu nome. Os tipos de dados e suas variáveis serão basilares para a montagem desta estrutura. Veja um exemplo:

Estrutura de matriz bidimensional:

[Código]

```
int aluno [3][4]; // 3 linhas e 4 colunas
```

[/Código]

É importante termos em mente que, na execução de uma programação, às operações e valores serão definidos pelos tipos de dados de uma variável. Quanto mais simples ela for, mais restrito será o tipo de armazenamento por vez. Da linguagem da programação você também irá perceber que os vetores podem ser nomeados de *arrays*, que são requisitados no processo de programação de computadores e na execução de processamentos de dados.

Os *arrays* também são considerados como variáveis subscritas e tabelas de dados em memória. Já em softwares como Java, eles são interpretados como tipos de referência. É importante entendermos também que a aplicabilidade das matrizes em 2D também está presente em testes relacionados à Inteligência Artificial. A eficácia de um projeto pode ser mensurada através da capacidade das redes neurais conseguirem fazer uma interpretação deste tipo de estrutura.

Matrizes tridimensionais

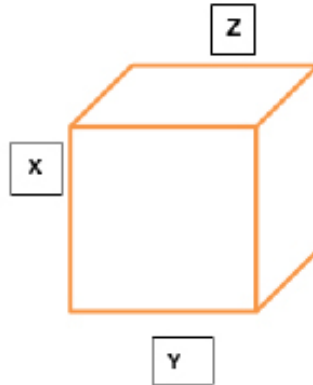
Uma matriz tridimensional, como o próprio nome diz, possui três dimensões: altura, largura e profundidade e cada elemento é relacionado através da posição que ocupa por linha, coluna e profundidade. A utilização desse tipo de matriz torna os algoritmos mais complexos, fazendo com que as bidimensionais sejam mais



requisitadas. Porém isso não anula a sua utilização dentro da resolução de problemas.

Na Figura 1, veja um exemplo de uma representação de matriz tridimensional com **x** linhas, **y** colunas e **z** profundidades.

Figura 1. Exemplo de matriz tridimensional.



Matriz [X] [Y] [Z] = N

Vetores e matrizes são semelhantes no aspecto de só poderem armazenar elementos do mesmo tipo de dado. O tamanho de uma matriz também não pode ser modificado após uma declaração. Portanto, a manipulação dos elementos deve ser realizada separadamente, pois não é possível realizar a mudança do conjunto ao mesmo tempo. Há uma espécie de sintaxe para realizarmos uma declaração de matriz em pseudocódigo, como podemos ver:

```
[Código]  
<nomeMatriz> : Matriz[1..<qtdeLinhas>, 1..<qtdeColunas>] De  
<tipo_de_dado>  
[/Código]
```

Ao utilizarmos variáveis simples, devemos seguir o conceito de que só podemos usar um valor a cada vez. Porém, as matrizes bidimensionadas possuem uma capacidade maior de informação, com menos trabalho ao processar. Retomando ao exemplo da Tabela 1, no caso do cálculo da média dos três alunos, temos ali uma só



variável contendo todos os valores das três notas. Isso seria representado da seguinte forma:

Exemplo da representação da média dos alunos:

```
[Código]
MD[1] = 6.5
MD[2] = 8.0
MD[3] = 6.5
[/Código]
```

Observe que o que muda apenas é a informação indicada dentro dos colchetes. Essa informação nós chamamos de índice, que é o local onde o elemento permanece. Já o elemento podemos conceituar como o conteúdo da matriz, que no exemplo está sendo representado pelas médias dos alunos. Exemplificando: no caso de MD[1] = 6.5, o número 1 é o índice, que é o local cujo elemento 6.5 está armazenado.

Informar a posição do elemento é fundamental para fazer uma operação de atribuição de valor, em elementos de uma matriz. No caso da utilização de matrizes, será definida a instrução vetor que indicará em a sua utilização. Sua síntese pode ser estruturada a partir da indicação da variável, valores inicial e final, do tamanho do vetor e se o vetor em questão irá utilizar valores reais, inteiros, lógicos ou caracteres. Podemos ainda exemplificar na seguinte maneira:

Exemplo de sintaxe de atribuição em matriz.

```
[Código]
{para atribuir}
<nomeMATRIZ>[<LINHA>,<COLUNA>]:=<VALOR>;
[/Código]
```

A leitura de uma matriz é processada passo a passo, um elemento por vez. A instrução de leitura é leia(), depois a variável e o índice. Como podemos ver:

Exemplo de sintaxe de atribuição em matriz.

```
[Código]
```



```
{para ler}  
<nomevariável> := <nomematriz>[<linha>,<coluna>];  
[/Código]
```

O processo de escrita de uma matriz é similar ao processo de leitura de seus elementos. Nessa circunstância deve-se colocar a instrução escreva () e em seguida a indicação da variável e seu índice. Vejamos um exemplo completo de algoritmo que declara vetor:

Exemplo de sintaxe de atribuição em matriz:

```
[Código]  
PROGRAMA UsoVetores;  
INICIO  
  Variável  
  int Valor[5], int Soma: INTEIRO;  
  
  ESCREVER("Digite quatro valores");  
  LER (VALOR[1]);  
  LER (VALOR[2]);  
  LER (VALOR[3]);  
  LER (VALOR[4]);  
  LER (VALOR[5]);  
  Soma := Valor[1]+ Valor[2]+ Valor[3]+ Valor[4] + Valor[5];  
  
  ESCREVER ("A soma dos números é ",intSoma);  
  
FIM.  
[/Código]
```

É possível ainda manipularmos os elementos da matriz, usando duas variáveis e uma repetição, como no exemplo a seguir:

```
[Código]  
#include<stdio.h>  
#include<stdlib.h>  
main()  
{  
  int aluno [3][5];  
  int i,j;  
  for(i=0;i<3;i++)  
  {  
    for(j=0;j<5;j++)
```



```
{  
printf("Aluno[%d] [%d]:aluno[i][j]\n",i,j, aluno[i][j]);  
}  
system("pause");  
}  
}  
[/Código]
```

Classificação dos elementos

A ordenação de elementos de uma matriz pode ser obtida através de alguns métodos aplicados. Um deles se baseia na ação de fazer uma comparação dos elementos, separadamente, com os demais que virão em seguida. A depender da ordem, seja ela crescente ou decrescente, se o elemento equiparado estiver menor (no caso da decrescente) ou maior (no caso da crescente) que o elemento que está em evidência, esta ordem será alterada e será adotada a organização em ordem alfabética.

Ao executarmos atividades que envolvam o uso de matrizes, nós podemos produzir tabelas com muitos dados sistêmicos. Em contrapartida, é importante pensarmos qual a melhor forma para conseguirmos extrair informações precisas ou mesmo buscar um dado específico sem perder muito tempo, no caso de tabelas muito grandes. Para otimizar as buscas, você pode usar os seguintes mecanismos: método sequencial e método de pesquisa binária.

O Método Sequencial é utilizado quando o usuário deseja localizar um determinado dado, começando do primeiro elemento seguindo até o último. Assim que a informação buscada é localizada, é demonstrada ao pesquisador. Mesmo não sendo o método mais eficaz, em termos de agilidade, ele é preciso nesse tipo busca, sobretudo quando há elementos fora de ordem na matriz utilizada.

Diferente da estrutura anterior, o método de pesquisa binária permite mais rapidez nas buscas, porém é necessário que a matriz tenha uma classificação prévia. É chamado de método binário, pois sua performance consiste em separar a lista em duas partes, com uma espécie de linha de divisão e buscar o dado pesquisado em cada



uma delas. Se a informação buscada já estiver na primeira parte da matriz, a busca é encerrada, eliminando automaticamente a segunda parte. Caso o que foi solicitado não seja encontrado desde a primeira divisão, a busca é refeita, subdividindo a lista e questionando em qual das partes há o que foi pedido. Para ficar mais claro, temos um exemplo de uma pesquisa binária:

Tabela 2. Exemplo de método de pesquisa binária.

Índice	Cores
1	Azul
2	Vermelho
3	Amarelo
4	Verde

Na Tabela 2 há a representação de uma matriz com quatro elementos. Vamos supor que a cor escolhida será o amarelo. Se utilizarmos o método de pesquisa binária, o número total de elementos será dividido por dois, desta maneira:

Primeira parte da primeira divisão:

Tabela 3. Exemplo de método de pesquisa binária.

Índice	Cores
1	Azul
2	Vermelho

Segunda parte da primeira divisão:

Tabela 4. Exemplo de método de pesquisa binária.

Índice	Cores
3	Amarelo
4	Verde

Caso a quantidade total dos elementos seja de um número ímpar, sempre se considera o quociente inteiro. Desse modo, uma das partes ficará com uma quantidade de elementos a mais. Após a



tabela ter sofrido a primeira divisão, deve-se averiguar em qual das partes está a informação do dado solicitado, que nesse caso é a cor amarelo. Percebendo que o amarelo está na segunda parte, então a primeira não será mais solicitada e a segunda sofrerá uma nova divisão, desta maneira:

Primeira parte da segunda divisão:

Tabela 5. Exemplo de método de pesquisa binária.

Índice	Cores
3	Amarelo

Segunda parte da segunda divisão:

Tabela 6. Exemplo de método de pesquisa binária.

Índice	Cores
4	Verde

Após essa segunda divisão, já é possível identificar que o amarelo está na primeira parte, portanto a segunda parte será desprezada. Se houvesse nesta parte a busca pelo nome de algum outro elemento que não correspondesse a nenhum desses dois, não seria apresentada nenhuma informação.

Veja outro exemplo de pesquisa binária:

[Código]

```
{Pesquisa binária}
Algoritmo ExemploPesqBinaria
Variáveis
  Valores: Vetor [1...8] de inteiro:
  ir, ia, aux: Inteiro; {usados na ordenação}
  Ir, if, it, itemPesq: Inteiro; {usado na pesquisa}
  Achou: Lógico;
  posAtual : Inteiro
01  Início
02  Para posAtual De 1 Até 8 passo 1 Faca
03    Escreva ("Digite um valor inteiro para a posição" +
04    posAtual);
    Leia (valores[posAtual]);
```



```
05  Fim_Para
06
    Escreva ("Digite o valor que deseja pesquisar:");
07  Leia(itemPesq);
08
09  Para ir De 1 Até 7 Passo 1 Faça
10    Para ia De ir+1 Até 8 Passo 1 Faça
11      Se (valores[ir] > valores [ia]) Então
12        aux := valores[ir];
13        Valores[ir] := valores[ia];
14        Valores[ia] := aux;
15  Fim_Se
    Fim_Para
16  Fim_Para
17
18  achou := F;
19  ii := 1; {índice inicial}
20  ii := 8; {índice final}
21  Enquanto ((ii <= if) .E. (achou = F)) Faça
22    It := (ii + if) / 2 ; {índice de teste}
23    Se (valores[it] = itemPesq) Então
24      Achou := V:
        Senão
25        Se (valores[it] > itemPesq) Então
26          {descarta da pesquisa os elementos da direita, até it}
          If := it - 1
27        Senão
28          {descarta da pesquisa os elementos da esquerda, até it}
29        ii := it + 1
30  Fim_Se
31  Fim_Se
32  Fim_Enquanto
    Se (achou = V) Então
33    Escreva ("O valor" + itemPesq +
34      "foi encontrado na posição" + it);
    Senão
35    Escreva ("O valor" + itemPesq +
      " não foi encontrado no vetor");
  Fim_Se
  Fim
```

[Código]

O algoritmo deste código traz de maneira detalhada como é a representação do modelo de pesquisa binária em código. Você pode perceber que o modelo realiza a leitura de oito valores inteiros,



alocando-os em um vetor (observe nas linhas de 1 a 4). Em seguida, há um requerimento de um valor que será explorado (conforme as linhas 5 e 6). Já entre as linhas 16 e 30, a pesquisa é realizada entre as linhas 31 e 35, por fim, é apresentado o resultado final ao testador.

Vale salientar que, para conseguirmos realizar essas ações, é importante termos os seguintes itens bem delimitados: os índices inicial, final e de teste. O índice inicial, como o nome sugere, indica o primeiro elemento da pesquisa, é representado pelo símbolo **II** e estará posicionado no primeiro elemento do vetor. O índice final indica o último elemento da pesquisa, é representando pelo **IF**. No início da busca, estará posicionado no último elemento. Já o índice de teste, indicado pelo símbolo **IT**, direciona qual será a posição seguinte a ser comparada. Ele estará no início da pesquisa próximo do elemento que mais se aproxima à parte central do vetor.



VisualG: matrizes

Desde o início, estamos tratando e aprendendo sobre os conceitos de algoritmos. Nós vamos utilizar, a partir de agora, uma ferramenta básica para ilustrar a aplicação destes conceitos: o VisualG. É importante ressaltar que existem também outras ferramentas adotadas para evidenciar os algoritmos em variáveis mais simples, entretanto, no caso das variáveis compostas, esse programa não é usado. A ideia a partir de agora é evidenciar, através de conceitos práticos, os fundamentos e aplicações referente ao uso de algoritmos.

Vamos tratar das matrizes dentro do programa VisualG e perceber que a importância da sua utilidade ocorre, por exemplo, no desenvolvendo de jogos, por conta da visão tridimensional que pode ser apresentada por meio de matrizes que foram inseridas. Isso indica que o uso de matrizes é fundamental para a computação gráfica atual. Só relembrando que as matrizes se caracterizam por apresentar um número maior de dimensões e podem ser representadas por expressões como esta:

```
[Código]
var
  m: vetor [1..3, 1..2] de inteiro
[/Código]
```

Tabela 8. Matriz 3x2.

	3
	4
7	

Isso indica, na prática, que teremos três dimensões de linhas e duas colunas representadas pela inserção de números inteiros, ao qual a variável irá apresentar. Importante lembrar a quantidade de dimensões fica a critério de cada usuário, ou seja, uma matriz pode apresentar duas, três ou mais linhas e colunas. Certamente você



pode se questionar: como eu consigo armazenar valores dentro das matrizes?

Bem, vamos imaginar um corpo de programa em que podemos inserir uma expressão $m = [l, c]$, onde “m” se refere a linha; “l” trata da primeira dimensão (linha) e “c” indica a segunda dimensão (coluna). De maneira sequencial, podemos adotar outros pares e montar uma tabela que representa a inserção desta matriz, como por exemplo:

[Código]

Início

$m[1,2] = 3$

$m[2,2] = 3$

$m[3,1] = 7$

[/Código]

De imediato é preciso inserir duas estruturas conforme o exemplo a seguir:

[Código]

Início

para $l <= 1$ até 3 faça

para $c <= 1$ até 2 faça

leia($m[l,c]$)

fimpara

fimpara

[/Código]

Uma estrutura representando o número de linhas (l) e a segunda simbolizando o número de coluna (c). é importante ressaltar que dentro destas expressões “para”, é possível inserir o comando leia ($m[l,c]$). Posteriormente, fecham-se os dois pares (fimpara) e é possível definir uma estrutura básica para montar valores e simular diversas alocações. Para que este conceito se torne mais prático e claro nós vamos, a partir de agora trazer alguns exemplos, utilizando as simulações realizadas dentro do programa VisualG.



Preenchimento de matrizes com valores via teclado

Vamos começar observando este corpo de comando extraído do programa VisualG:

```
[Código]
algoritmo "lerMatriz"
var
    mat: vetor [1..3, 1..2] de Inteiro
    l, c: Inteiro
início
    para l <- 1 ate 3 faça
        para c <- 1 ate 2 faça
            Escreva ("Digite um valor: ")
            Leia (mat [l,c])
        fimpara
    fimpara
finalgoritmos
[/Código]
```

Você irá perceber nesse exemplo, de imediato, que vamos ter uma matriz formada por três linhas e duas colunas, representada pela inserção de números inteiros. Posteriormente foram inseridas as duas estruturas (uma dentro da outra) que simbolizam as linhas e colunas, além de delimitar a sua extensão. Por fim vamos informar por meio do comando "ESCREVA" um valor que podemos atribuir e o comando LEIA, para que as variáveis sejam reconhecidas.

Ao executar o algoritmo vamos observar a área de variáveis composta por um conjunto de matrizes com seus respectivos valores. A princípio, é possível visualizar o valor atribuído à linha 1/ Coluna 1, ou seja, se eu atribuir um valor qualquer, ele será alocado nesta posição. A partir dessa execução, na tela o sistema irá atribuir os valores seguintes a cada posição até completar a área.



Certamente você vai perceber, ao realizar outras ações, onde é possível apresentar a matriz disposta na tela. Observe, novamente, o corpo de comando extraído do programa VisualG.

[Código]

```
algoritmo "lerMatriz"
```

```
var
```

```
    mat: vetor [1..3, 1..2] de Inteiro
```

```
    l, c: Inteiro
```

```
início
```

```
    para l <- 1 ate 3 faça
```

```
        para c <- 1 ate 2 faça
```

```
            Escreva ("Digite o valor da posição [", l, ", ", c, "]: ")
```

```
                Leia (mat [l,c])
```

```
            fimpara
```

```
        fimpara
```

```
            para l <- 1 ate 3 faça
```

```
                para c <- 1 ate 2 faça
```

```
                    Escreva (mat [l,c])
```

```
                fimpara
```

```
            EscrevaL ( )
```

```
        fimpara
```

```
finalgoritmos
```

```
[/Código]
```

Com o objetivo de facilitar a leitura, nós realizamos uma alteração no comando "Escreva". E o que isto significa? Bem, você vai notar que os valores de todas as posições estarão dispostos e o usuário terá uma noção maior do que está acontecendo. Diante disso, é possível estabelecer o comando "EscrevaL ()" para a exibição da matriz conforme se visualizou entre os comandos "fimpara", ficando a critério do usuário o tamanho dos espaços entre os valores.

Leitura de matriz e demonstração dos pares

Na segunda situação prática que iremos observar o uso, por exemplo, de uma matriz de dimensão 3x3 (terceira ordem) e apresentar quais destes valores são pares no momento de exibir na tela. Vejamos, novamente, um corpo de programa extraído do VisualG.

[Código]



```
algoritmo "lerMatriz"
var
    valores: vetor [1..3, 1..3] de Inteiro
    l, c: Inteiro
início
    paral <- 1 ate 3 faça
        parac <- 1 ate 3 faça
            Escreva ("Digite o valor da posição [", l, ", ", c, "]: ")
            Leia (valores [l,c])
        fimpara
    fimpara
    paral <- 1 ate 3 faça
        parac <- 1 ate 3 faça
            Escreva (valores [l,c])
        Fimpara
    EscrevaL ( )
fimpara
finalgoritmos
[/Código]
```

Como você consegue observar, ao comando "EscrevaL ()" é possível novamente exibir a matriz na tela, agora por meio de uma dimensão de terceira ordem, ficando a critério do usuário definir o tamanho dos espaços entre os valores. Porém, para informar os números pares que compõem uma matriz, é preciso inserir novas informações, conforme se observa no corpo do programa:

```
[Código]
algoritmo "lerMatriz"
var
    valores: vetor [1..3, 1..3] de Inteiro
    l, c: Inteiro
início
    paral <- 1 ate 3 faça
        parac <- 1 ate 3 faça
            Escreva ("Digite o valor da posição [", l, ", ", c, "]: ")
        Leia (valores [l,c])
        fimpara
    fimpara
    paral <- 1 ate 3 faça
        parac <- 1 ate 3 faça
            Se (valores [ l, c] % 2 = 0 entao
                Escreva (" (" , valores [ l, c ]: 2, ")")
            senao
```




Escreva (valores [l,c])

fimpara
EscrevaL ()

fimpara
fimalgoritmos
[/Código]

Um detalhe importante é a existência de outros comandos que podem ser inseridos na tela VisualG para realizar outras simulações em relação às matrizes, principalmente na busca total das amostras de números de uma matriz, por exemplo.



DICA

No período de testes, o VisualG facilita a inserção de valores através do uso de um recurso presente na barra de ferramentas do programa (gerar valores aleatórios), que consegue inserir valores aleatórios no teclado. Ele pode simular e poupar o usuário de sair digitando uma infinidade de valores, por ser uma atividade extremamente cansativa e repetitiva.

Criação de uma matriz identidade de terceira ordem

Vamos visualizar de fato os conceitos da matemática de matrizes inseridos nos algoritmos. É importante ressaltar que quando nos referimos a essa modalidade de matriz estamos tratando de uma matriz 3x3 que apresenta os seguintes aspectos:

Tabela 7. Matriz 3x3.

1	0	0
0	1	0



0	0	1
1		
2		
3		
1	2	3

Esses valores que acabamos de atribuir são denominados de “diagonal principal”, pois apresentam igualdade nas linhas e colunas visualizadas, ou seja, o algarismo 1 ocupa as posições [1,1]; [2,2]; [3,3], respectivamente. Portanto, em uma matriz identidade é possível observar que a diagonal principal será formada pelo número 1 e as demais pelo número 0, completando tabela. O programa VisualG não vai conseguir realizar a leitura, pois ele é o responsável por gerar esta matriz simplesmente ao clicar a tecla F5.

```
[Código]
algoritmo “matrizIdentidade”
var
    mID: vetor [1..3, 1..3] de Inteiro
    l, c: Inteiro
início
    para l <- 1 ate 3 faça
        para c <- 1 ate 3 faça
            Se (l = c) então
                mID[l, c] <- 1
            senão
                mID[l, c] <- 0
        fimse
    fimpara
fimpara

Fimalgoritmos
[/Código]
```

Podemos realizar algumas considerações:

Se, por ventura, o valor atribuído à linha (l) for igual ao da coluna (c), será atribuído um valor 1, pois ele representa a diagonal principal. Caso contrário o valor atribuído será 0.



Para visualizar esta matriz na tela do VisualG, basta inserir os comandos complementares.

[Código]

```
algoritmo "matrizIdentidade"
var
    mID: vetor [1..3, 1..3] de Inteiro
    l, c: Inteiro
início
    para l <- 1 ate 3 faça
        para c <- 1 ate 3 faça
            Se (l = c) então
                mID [l, c] <- 1
            senão
                mID [l, c] <- 0
            fimse
        fimpara
    fimpara
    para l <- 1 ate 3 faça
        para c <- 1 ate 3 faça
            Escreva (mID [l,c]:)
        Fimpara
        Escreva L ()
    fimpara
Fimalgoritmo
[/Código]
```

Preenchimento de matriz de quarta ordem

Vale lembrar que essa matriz identidade pode ser alterada para a dimensão escolhida pelo usuário, ou seja, ao invés de uma matriz 3 x 3, para uma dimensão maior como 4 x 4, por exemplo.

[Código]

```
algoritmo "valoresMatriz"
var
    m: vetor [1..3, 1..3] de Inteiro
    l, c: Inteiro
início
    para l <- 1 ate 4 faça
        para c <- 1 ate 4 faça
```



```
Escreva ("Digite o valor da posição [“, 1, “, “, c, “]: “)
Leia (m[l,c])
    fimpara
fimpara
fimalgoritmos
[/Código]
```

Da mesma maneira que nos ocorreu outros exemplos, é possível estabelecer o corpo da programação e inserir as informações principais. A partir daí, nós vamos inserir algumas atribuições:

1. A soma dos valores da diagonal principal:

```
[Código]
algoritmo “valoresMatriz”
var
    m: vetor [1..3, 1..3] de Inteiro
    l, c, sDP: Inteiro
início
sDP ← - 0
    para l ← 1 até 4 faça
        para c ← 1 até 4 faça
            Escreva ("Digite o valor da posição [“, 1, “, “, c, “]: “)
            Leia (m[l,c])
            Se (1 = c) então
                sDP ← sDP + m[l, c]
            FimSe
        fimpara
    fimpara
    EscrevaL (“ A soma dos valores da Diagonal principal e”, sDP)
fimalgoritmos
[/Código]
```

2. Produto dos valores da segunda linha:

```
[Código]
algoritmo “valoresMatriz”
var
    m: vetor [1..3, 1..3] de Inteiro
    l, c, sDP, p2L : Inteiro
início
sDP ← - 0
    p2L ← - 1
```



```
    paral <- 1 ate 4 faca
    parac<- 1 ate 4 faca
    Escreva ("Digite o valor da posição [“, 1, “, “, c, “]: “)
Leia (m[l,c])
Se (1 = c) então
sDP<- sDP + m[l, c]
FimSe

        fimpara

    paral <- 1 ate 3 faca
        parac<- 1 ate 3 faca
            Escreva (mID [l,c]:)
        Fimpara
        Escreva L ()
    fimpara
    parac<- 1 ate 4 faca
    p2L<- p2L + m[2, c]
    fimpara
    EscrevaL (“ A soma dos valores da Diagonal principal e”, sDP)
EscrevaL (“ O produto dos valores da segunda linha e”, p2L)
finalgoritmos
[/Código]
```



Modularização de algoritmos

Para podermos lidar com uma situação de muita complexidade, é interessante pensarmos em um esquema prático, para resolvê-la da melhor maneira. Um passo importante é não enxergarmos apenas a situação por completo, mas cada uma de suas partes, com os seus problemas particulares. Com essa estrutura, nós lidamos com um problema por vez, o que permite que as resoluções também ocorram a cada fase.

Quando falamos de programação, a situação é bem similar. Há algoritmos que são de alta complexidade e exigem do usuário maior dedicação. Porém, com a técnica de uma programação estruturada, em que há uma espécie de “divisão de tarefas”, é possível utilizar blocos, como estruturas fundamentais para solucionar situações mais difíceis, o mesmo que transformar um grande problema em diversos problemas menores.

Um algoritmo considerado de alta complexidade pode ser subdividido em partes, que trabalham em conjunto. Na linguagem de programação, essas partes são conhecidas como módulos, que reúnem funções com objetivos similares e facilitam a manutenção e desenvolvimento de um sistema. Ao utilizar os módulos, os usuários de uma função podem dedicar-se em atividades diversas, porém do mesmo programa.

Diante disso, podemos definir que a modularização é justamente esta ação de dividir o algoritmo por módulos. Essa é uma maneira de trabalhar o código de uma forma que o deixe mais simples, que permita que uma das partes do aplicativo não seja repetida de maneira desnecessária e que as soluções aplicadas possam ser utilizadas novamente. Numa programação estruturada, a definição de modularização também é relacionada, uma vez que para o desenvolvimento de blocos organizados as funções desempenham uma atividade crucial.



Vantagens do uso da modularização

- Facilidade na descrição: com o algoritmo em partes, é possível escrever a solução para cada um, em particular, o que é menos complexo que fazer com o programa completo, de uma única vez;
- Facilidade na leitura: por estar em módulo, a leitura do algoritmo também será feita separadamente, de maneira mais simples, permitindo melhor compreensão do caso, principalmente se for por lido por alguém que não o desenvolveu;
- Facilidade no entendimento: somente fazendo a leitura dos módulos já é possível entender o algoritmo como um todo;
- Mais agilidade: a separação em módulo permite ainda mais aproveitamento do tempo e agilidade no processo, pois poderemos requisitá-lo de qualquer lugar, independentemente da quantidade de vezes, sem necessariamente precisar reescrever a atividade por várias vezes;
- Linguagem abrangente: não é preciso mudar o módulo caso ele seja utilizado em outro algoritmo, pois sua linguagem se estende e pode ser lida por outros sistemas.

Para entendermos o processo de modularização do algoritmo, é preciso ter em mente que o que permite a sua execução são as sub-rotinas. As sub-rotinas desempenham sua atividade fora do corpo principal do algoritmo e possuem dois tipos: procedimentos e funções. Veremos esses aspectos de maneira mais detalhada, com seus conceitos e aplicabilidade no sistema.

Procedimentos

O procedimento é um tipo de sub-rotina definido como um conjunto de códigos no qual é possível reunir as orientações de uma atividade



específica. Ele irá conter início e fim, sendo identificado através de um nome que preferencialmente tenha alguma relação com as atividades realizadas. Por permitir uma interação direta com o programador, o procedimento é usado de maneira corriqueira. Quando falamos em reaproveitar soluções, esse tipo de sub-rotina é visto como facilitador, pois pode ser usado em mais de um algoritmo, com a mesma função, sem perder a eficácia.

Além disso, também é importante entender que este módulo é declarado em um bloco próprio e dentro dele é informado todos os comandos que condizem com a atividade para qual foi desenvolvido. A declaração também deve seguir um padrão, que será executado antes do demarcador inicial do corpo do algoritmo e posterior ao bloco de variáveis. Uma vez declarado, o procedimento poderá ser chamado em qualquer área do sistema. Em português estruturado, esta é a sua sintaxe básica:

```
[Código]
Procedimento<nome do procedimento>
var
    <variáveis>
inicio
    <instruções>
Fim_Procedimento
[/Código]
```

Humberto Beneduzzi e João Metz, no livro *Lógica e linguagem de programação: introdução ao desenvolvimento de software*, lançado em 2010, elaboraram um exemplo utilizando o procedimento, que será um norteador para que você visualize como ele fica disposto em pseudocódigo. Veja:



[Código]

```
{Exemplo de procedimento}  
Algoritmo ExemploProcedimento  
Variáveis  
  valores: Vetor[1..10] De Inteiro;  
  contador : Inteiro;
```

```
Procedimento LerValores()
```

```
Início
```

```
  Para contador De 1 Até 10 Passo 1 Faça
```

```
    Escreva ("Digite um valor para a posição" + contador +  
      " : ");
```

```
    Leia(valores[contador]);
```

```
  Fim_Para
```

```
Fim_Procedimento
```

```
Procedimento EscreverValores()
```

```
Início
```

```
  Para contador De 1 Até 10 Passo 1 Faça
```

```
    Escreva("O elemento da posição" + contador +  
      "vale" + valores[contador])
```

```
  Fim_Para
```

```
Fim_Procedimento
```

```
Início
```

```
  LerValores();
```

```
  EscreverValores();
```

```
Fim
```

[/Código]

Na representação desse algoritmo foram declarados dois procedimentos, como você pode observar. Um foi para ler e o outro para escrever os valores. As chamadas ficaram no corpo principal. Caso fosse necessário escrever os valores por várias vezes, não seria preciso repetir o bloco de instruções, mas apenas inserir várias chamadas ao procedimento.

Funções

As funções são trechos de códigos, com início e fim, identificados por um nome, que também auxiliam na simplificação de um problema



no algoritmo. Porém, diferente do procedimento, as funções retornam um valor ao fim de sua execução. Esse valor que retorna, por sua vez, pode ser enviado à uma tela ou ser usado em uma atribuição. Veja um exemplo de sua aplicação. Observe que os comandos são semelhantes aos do procedimento.

[Código]

```
{Exemplo de função}
01  Algoritmo ExemploFuncao
02  Variáveis
03      valorVenda, valorComissao : Real ;

      Retorna um valor de comissão referente ao valor da venda.
      Regras: 5% para vendas >= 500 e 7.5 % para vendas < 500}
04  Função CalcularComissao() : Real
05  Início
06      Se(valorVenda>=500) Então
07          CalcularComissao := valorVenda * 0,05;
08      Senão
09          CalcularComissao := valorVenda * 0,075;
10      Fim_Se
11  Fim_Função

12  Início
13      ValorVenda := 0;
14      Escreva("Cálculo de comissão sobre venda");
15      Repita
16          Escreva ("Digite o valor da venda ou 0 para sair: ");
17          Leia (valorVenda);
18          valorComissao := CalcularComissao();
19          Escreva("O valor da comissão é: " + valorComissao);
20          Até_Que(valorVenda = 0)
21          Escreva ("Fim de Execução");
22  Fim
```

[/Código]

Fonte: BENEDUZZI; METZ, 2010.

Esse código trata de quanto será o valor da comissão de vendas de um determinado local. O trecho entre as linhas de 4 e 11 mostra a declaração da função que irá fazer o cálculo da devida comissão sobre as vendas. Na linha 18, a função é apresentada com a variável "valorComissao" relacionada ao seu valor. Você pode perceber



também que, nesse caso, as funções não precisam ficar apenas em uma linha separada, elas podem ser inseridas dentro das expressões. Esse mecanismo permite que o número de variáveis dentro do algoritmo seja diminuído.

Porém, há casos e casos e a aplicação pode variar de acordo com o que venha a ser solicitado. Portanto, caso você perceba que a “economia de variáveis” não será tão vantajosa, não tema em retornar o valor dado pela função em outros pontos do algoritmo. Analise que tipo de problema deseja resolver e veja qual o melhor método a ser adotado.

Segue outro exemplo de algoritmo usando as funções:

[Código]

```
{Exemplo funções 2}
01 Algoritmo ExemploFuncoes2
02 Tipos
03   RegAluno: Registro
04     nome: Caractere;
05     nota1, nota2, media : Real;
06   Fim_Registro
07 Variáveis
08   alunos : Vetor[1..40] De RegAluno;
09   notasTemp : Real;
10   cont, qtdeAprovados : Inteiro;

11 Procedimento LerAlunos()
12 Início
13   Escreva ("Informe os dados dos alunos");
14   Para cont De 1 Até 40 Passo 1 Faça
15     Escreva ("Informe o nome do aluno" + cont);
16     Leia(alunos[cont].nome);
17     Escreva("Informe a nota 1:");
18     Leia(alunos[cont].nota1);
19     Escreva("Informe a nota 2:");
20     Leia(alunos[cont].nota2);
21     alunos[cont].media :=
22   Fim_Para
23 Fim_Procedimento
```



```
24  Função CalcularMediaGeral() : Real
25  Início
26    notasTemp := 0;
27    Para cont De 1 Até 40 Passo 1 Faça
28      notasTemp := notasTemp + alunos[cont].media;
29    Fim_Para
30    CalcularMediaGeral := notasTemp / 40;
31  Fim_Função

32  Função ContarAprovados() : Real
33  Início
34    qtdeAprovados := 0;
35    Para cont De 1 Até 40 Passo 1 Faça
36      Se(alunos[cont].media >= 7) Então
37        qtdeAprovados := qtdeAprovados + 1;
38      Fim_Se
39    Fim_Para
40    ContarAprovados := qtdeAprovados;
41  Fim_Função

42  Início
43    LerAlunos();
44    Escreva ("A média geral da turma é: "CalcularMediaGeral
    ());
45    Escreva ("A quantidade de alunos aprovados na turma é: "
    + ContarAprovados());
46  Fim
[/Código]
```

Nesse segundo exemplo, que é um pouco mais elaborado. É possível perceber se trata de um algoritmo para calcular a média dos alunos de uma determinada turma. Nele, foram utilizadas as duas sub-rotinas. Nas linhas de 11 a 23 foi aplicado o procedimento, já que nesta parte não necessitava de um valor de retorno. Entre as linhas 24 e 31 foi preciso o uso da função, pois seria calculada a média da turma e o valor de retorno seria enviado à tela, conforme solicitado na linha 44. Há uma função solicitada para calcular a quantidade de alunos aprovados (entre as linhas 32 e 41) e a função é chamada na linha 45 de novo, para enviar o valor à tela.

No programa, existe uma área a qual a variável será criada. É nesse espaço que irá ser definida a sua criação e visibilidade. Esta



área é denominada de **escopo**. Dentro deste conceito, temos duas classificações: **variáveis globais** e as **variáveis locais**.

As variáveis globais são criadas desde o início do algoritmo e podem ser acessadas em qualquer ponto dele. As variáveis locais, como o nome diz, são um pouco mais restritas e só podem acessadas de dentro da sub-rotina, ou seja, na sua própria estrutura, fora do corpo principal do algoritmo.

Veja como elas ficam dispostas:

[Código]

```
{Exemplo de variáveis locais x globais}
01  Algoritmo ExemploEscopoVar
02  Variáveis
03    valorA, valorB : Inteiro;

04  Função EscreverVarLocal() : Inteiro
05  Variáveis
06    valorB, valorC : Inteiro;
07  Início
08    valorB := ValorA * 2;
09    Escreva(ValorB);
10    valorC := 7;
11  Fim_Função

12  Início
13    valorA := 5;
14    valorB := 3;
15    Escreva(valor B) {escreve "3"}
16    Escreva(EscreverVarLocal); {escreve "10"}
17    Escreva(valorB) {escreve "3"}
18    Escreva(valorC) {gerará um erro, por valorC não
    existe aqui}
19  Fim
```

[/Código]

Nesse exemplo, você perceberá como estão dispostas as variáveis globais e locais. Nas linhas 1 e 2 estão declaradas as variáveis globais, representadas pelo valor A e valor B. Nas linhas 5 e 6, você



verá que são declaradas as variáveis locais Valor B e Valor C, dentro da Função EscreverVarLocal. A variável valor A está em qualquer lugar do algoritmo, portanto é global.

Há duas variáveis com o mesmo nome de Valor B, uma é global e a outra é local. A Valor B local só será declarada quando acessarmos dentro da sub-rotina (linha 16). Se acessarmos dentro do corpo do algoritmo, teremos o valor da global (linha 15). Já a Valor C é uma variável local, disponível apenas dentro da função EscreverVarLocal. Se ela for acessada no corpo do algoritmo, haverá um erro (linha 18).

As variáveis locais podem trazer benefícios no sentido de deixar o algoritmo livre de tantas informações. Como as globais podem ser encontradas em qualquer parte do corpo do algoritmo, é preciso verificar em que parte ela é utilizada, para que, se for fazer algum tipo de alteração, não interfira no funcionamento de todo o sistema. Isso não ocorre com as locais, pois só existem em pontos específicos do sistema.

Aprendemos que os processos de modularizar os algoritmos facilitam na sua compreensão, pois divide um grande problema em pequenos problemas, melhorando o seu entendimento e capacidade de resolução. As sub-rotinas, que permitem esse processo, tornam mais rápidas as buscas por dados específicos, bem como permitem que a manutenção do programa seja feita separadamente, sem interferir nos demais dados.

VisualG: Modularização

Vamos dar continuidade ao entendimento na área de algoritmos através de um tema novo: as rotinas (modularização), que se referem às funções e procedimentos aos quais os algoritmos são submetidos. Bem, quando nos referimos ao termo “rotina” pode fazer uma analogia a um evento que se repete constantemente. Para ficar mais claro imagine, por exemplo, um moedor de carne que precisa de uma série de procedimentos para executar a sua função.



EXEMPLIFICANDO

Modularização, por definição, é o agrupamento de um conjunto de ações que tenham um objetivo específico.

Como exemplo, podemos citar uma casa. A casa como um todo tem sua função principal, mas internamente os cômodos da casa (módulos) também têm a sua função específica que implementam a função principal da casa. E as pessoas podem entrar em cada cômodo e sair deles.

Entretanto vale ressaltar que os procedimentos de execução e preparo são idênticos e repetitivos, o que indicará o surgimento de uma rotina. Trazendo esse entendimento para um sistema de algoritmos é preciso entender que não há necessidade de escrevê-lo por diversas vezes no programa principal. Diante desse breve exemplo, vamos apresentar algumas situações práticas que podem ser aplicadas dentro do programa VisaulG.

Detectores

É possível visualizar no programa a seguir um programa definido como detectores, utilizado para apurar o peso de pessoas ou objetos. E de que maneira ele funciona? Imagine uma tela onde foram inseridas informações referentes aos pesos de um grupo de pessoas, onde um determinado indivíduo foi escolhido como o mais pesado através da verificação feita pelo sistema. Essa observação está relacionada à descrição do código visualizado a seguir:

```
[Código]
Algoritmo: "DetectorPesado"
Var
    I: Inteiro
    N, Pesado: Caractere
    P, Mai: real
início
    LimpaTela
```



```
EscrevaL( "-----")
-----")
EscrevaL ( "DETECTOR DE PESADO")
EscrevaL ( "Maior Peso até agora: " Mai, Kg")
EscrevaL( "-----")
-----")
Para I< - 1 até 5 faça
    Escreva ("Digite o nome : ")
    Leia ( N )
    Escreva ("Digite o peso de ", N, ": ")
    Leia ( P )
    Se( P>Mai) então
        Mai <- p
        Pesado < - N
    FimSe
    LimpaTela
    EscrevaL ( "-----")
    EscrevaL ( "DETECTOR DE PESADO")
    EscrevaL ( "Maior Peso até agora: " Mai, Kg")
    EscrevaL( "-----")
    FimPara
    LimpaTela
    EscrevaL ( "-----")
    EscrevaL ( "DETECTOR DE PESADO")
    EscrevaL ("Maior Peso até agora: " Mai, Kg")
    EscrevaL ( "-----")
    EscrevaL ( " A pessoa mais pesada foi", Pesado, ", com", Mai, "quilo".)
finalgoritmo
[/Código]
```

Como você pode observar, os comandos que se iniciam com termos "LimpaTela" e se encerra com "EscrevaL" foi repetido por diversas vezes no código, o que significa dizer que é possível transformá-lo em um procedimento, o que torna a codificação mais simplificada, conforme se observa seguir:

```
[Código]
Algoritmo: "DetectorPesado"
Var
    I: Inteiro
    N, Pesado: Caractere
    P, Mai: real
Procedimento Topo ( )
```




```
Início
LimpaTela
    EscrevaL( "-----"
-----" )
    EscrevaL ( "DETECTOR DE PESADO")
    EscrevaL ("Maior Peso até agora: " Mai, Kg")
    EscrevaL( "-----"
-----" )
FimProcedimento
início
Topo ( )
Para I< - 1 até 5 faça
    Escreva ("Digite o nome : ")
    Leia ( N)
    Escreva ("Digite o peso de ", N, ": ")
    Leia ( P)
    Se( P>Mai) então
        Mai <- p
        Pesado <- N
    FimSe
Topo ( )
FimPara
Topo ( )
finalgoritmo
[/Código]
```

Como você pode observar, nós copiamos uma sequência de linhas citadas anteriormente e inserimos dentro do espaço do procedimento adotado. Percebemos que podemos substituir este conjunto de linhas pelo termo “topo”, adotado anteriormente. Além de simplificar o código adotado, nós vamos perceber que sempre que for necessário adotar algum processo, a codificação vai retornar sempre ao procedimento inicial estabelecido.

Passagem de parâmetros

Quando tratamos desse tema é possível subdividi-lo em alguns aspectos: um destes aspectos é o “parâmetros por valor”. Ele funciona da seguinte maneira: quando eu quero que algum procedimento seja executado, eu indico o que precisa ser realizado, como nos passos pra verificar o peso das pessoas. Entretanto, alguns procedimentos



são executados por meio de parâmetros, ou seja, quando estabelecemos um limite para executarmos uma determinada ação. Vejamos um procedimento simplificado:

```
[Código]
Procedimento Soma (C, D: Inteiro)
Início
    EscrevaL ("Recebi o valor", C)
    EscrevaL ("Recebi o valor", D)
    EscrevaL ("A soma vale ", C + D)
FimProcedimento
[/Código]
```

Como podemos notar, os termos “C” e “D” são os parâmetros, pois passam por um determinado procedimento que começa a partir da inserção dos termos “início”. Após a inserção de comandos de entrada, é possível finalizar a codificação. Utilizando o VisualG, este procedimento pode ser inserido da seguinte maneira:

```
[Código]
Algoritmo "FazSomaProcedimento"
Var
X, Y: Inteiro
Procedimento Soma (C, D: Inteiro)
Início
    EscrevaL ("Recebi o valor", C)
    EscrevaL ("Recebi o valor", D)
    EscrevaL ("A soma entre os dois e", C + D)

FimdeProcedimento
Início
    X<- 5
    Y<- 3
    Soma (X, Y)
Fimalgoritmo
[/Código]
```

Primeiramente é preciso inserir dois parâmetros que vamos chamar de X, Y. Posteriormente vamos atribuir valores a estes parâmetros e estabelecer uma soma entre estes. Lembrando que a soma não é comando existente, portanto ele precisa ser criado em



forma de procedimento onde X, Y serão representados pelos termos C, D. após a introdução dos comandos de “EscrevaL” dentro procedimento e executá-lo.

Verificação dos termos

Observe a codificação a seguir:

```
[Código]
Algoritmo: “ParouImparProcedimento”
Var
N Inteiro
ProcedimentoParouImpár (V: Inteiro)
Início
Se ( $V \% 2 = 0$ ) então
EscrevaL(“O numero ‘, V, “ e PAR”)
Senão
EscrevaL(“O numero ‘, V, “ e IMPAR”)
FimSe
FimdeProcedimento
Início
EscrevaL (“Digite um número:”)
Leia (N)
ParouImpar (N)
Fimalgoritmo
[/Código]
```

Inserimos uma variável “n” inteira e realizar a sua leitura. É preciso inserir comandos no início e adotar um procedimento denominado de “Par ou Ímpar”, que verificará este atributo na variável “N”. Posteriormente, é necessário adotar comandos do início ao fim do procedimento e posteriormente ser executado, lembrando que o procedimento sempre retorna ao início para ser executado.



SINTETIZANDO

Observamos que as matrizes são técnicas de programação que possibilitam o agrupamento de várias informações em uma mesma variável, respeitando o uso de dados semelhantes, denominados de estrutura de dados homogênea.

Outro aspecto interessante tratou do uso da ferramenta básica VisualG, que é um programa adotado para apresentar a funcionalidade dos algoritmos compostos apesar da existência de outras ferramentas adotadas para evidenciar os algoritmos em variáveis mais simples.

Vimos que um algoritmo complexo pode ser subdividido em etapas conhecidas como módulos, que reúnem funções similares e auxiliam na manutenção e geração de um sistema. Ao adotar os módulos, os usuários de uma função podem atuar em atividades variadas do mesmo programa.

As rotinas (modularização) foram mais um ponto interessante, pois tratou das funções e procedimentos aos quais os algoritmos são submetidos. Sabendo que rotina pode fazer uma analogia a um evento repetido constantemente e que precisa de uma série de procedimentos para executar a sua função.



Objetivos

1. Apresentar as características referentes aos processos de programação, evidenciando as suas técnicas e aspectos principais;
2. Abordar aspectos das linguagens de programação mais usuais, evidenciando as suas funcionalidades, classes e estilos.



Processo de programação

A programação é conceituada como o processo para desenvolver um programa de computador. Isso envolve a escrita e a manutenção de um programa utilizando uma linguagem específica. Portanto, é possível afirmar que esse processo está relacionado aos tipos e conceitos básicos da programação – que veremos ao decorrer deste assunto de maneira mais detalhada.

Técnicas de desenvolvimento de programas

Nós já vimos que há uma grande vantagem em buscarmos boas estratégias para a solução de um problema de grande complexidade. Nessa circunstância, é importante não pensarmos no contexto como um todo, mas em suas partes, transformando um grande problema em pequenos problemas, que são melhores para trabalhar e solucionar.

Tratando-se de programação, quando um problema complexo é subdividido em partes, cada uma delas acaba por se transformar em um fragmento de código, denominado como **módulo**. Portanto, a programação modular é justamente essa ação de dividir um programa mais amplo em subprogramas.

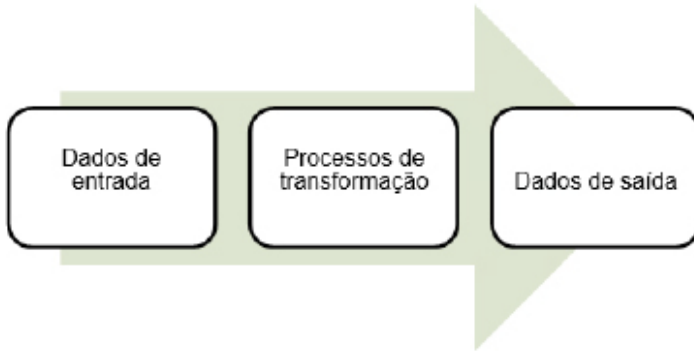
Vale ressaltar que dividir o programa não se restringe apenas em transformar o código em pequenos pedaços, sem distinção, pois o processo modular em si também envolve uma análise cuidadosa de cada ação que o programa vai desenvolver. É comum acontecer um refinamento de código, que é quando um módulo precisa sofrer uma nova fragmentação mesmo após o programa todo já ter sido dividido.

É importante que você tenha o cuidado, neste processo de “cortar” um programa em pedaços, em perceber como está o desempenho dos módulos. Cada módulo precisa ser o mais independente possível do outro, ou seja, possuir um fraco acoplamento, assim como precisa possuir uma forte coesão, que é não se romper com facilidade de alguma parte do mesmo.



Os módulos podem ter em sua formação o código-fonte ou rotinas executadas por meio da parte principal deste programa. Esses códigos precisam ainda ter uma conexão formada de um único ponto de entrada e saída, e sua maneira de exercitar as atividades deve ser independente das demais partes do programa. Veja uma representação na Figura 1.

Figura 1. Representação de uma programação modular.



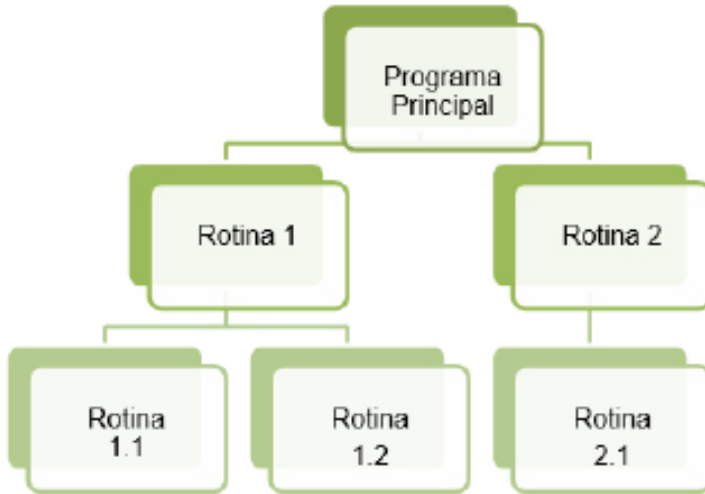
Fonte: MANZANO, 2000, p. 15.

O código-fonte serve para a entrada de um compilador, que contém uma linguagem de programação e a rotina, que está relacionada com os procedimentos e funções. Mesmo não havendo um padrão para realizar esse processo de modularizar o código, é importante que você saiba que há duas técnicas vigentes: **top-down** e **bottom-up**.

Na técnica top-down, que em tradução simples significa “de cima para baixo”, o problema é visto em sua magnitude e, posteriormente, são identificadas todas as partes que devem integrar a sua composição. Somente após essa análise é que ele é dividido em partes menores; portanto, parte-se do conceito macro até o refinamento de conceitos menores dentro do próprio programa.



Diagrama 1. Representação de um programa utilizando método top-down



No Diagrama 1, vemos a representação de um programa utilizando sub-rotinas e o método top-down. Nesses casos, por exemplo, temos uma rotina específica para cada função: uma para processamento, outra para saída dos dados e assim por diante.

Quando falamos da técnica bottom-up, por sua vez, tem o conceito contrário do top-down, pois parte-se da visão micro de cada pedaço, em que há mais detalhamento, direcionando para o conceito mais amplo, por meio do agrupamento dessas partes. Esse tipo de método, que significa “de baixo para cima”, é indicado quando é necessário conseguir programas precisos.

Você já percebeu que esse método possibilita que módulos sejam criados e utilizados em programas variados. Contudo, é preciso que tenhamos atenção a alguns pontos importantes para que consigamos obter êxito na solução dos problemas apresentados. Os módulos podem ser testados de maneira individual, o que possibilita também que a correção dos erros seja feita da mesma maneira, o que otimiza a execução da atividade.

Por outro lado, justamente por haver essa separação, pode ocorrer certa resistência na ligação dos módulos que não foram desenvolvidos pelo mesmo programador, assim como pode ter uma



necessidade a mais de criar outros programas para testar esses módulos, o que requer maior disponibilidade do usuário.

A **programação estruturada** é definida pela utilização de blocos como estrutura básica para a construção de um sistema. Essas estruturas são ampliadas até que alcancem o nível necessário para resolver um problema específico. É interessante tomarmos conhecimento de que cada um desses blocos é constituído por orientações que serão desenvolvidas de maneira conjunta, justamente por estarem relacionadas.

Com essas estruturas, é possível determinar uma sequência de operações que serão desempenhadas sistemicamente. Isso faz com que a programação seja mais assertiva e os erros sejam detectados de maneira rápida, facilitando a sua correção. Por terem entradas e saídas simples, as operações são mais direcionadas, sem o perigo de haver duplos resultados.

O código final, que é obtido por meio dessa técnica, é compreendido facilmente, não somente por quem o desenvolveu. Outros benefícios com o uso dessa programação é a inserção de estruturas em módulos diferentes e a ampliação da produtividade dos desenvolvedores, assim como é permitida uma padronização por conta da redução da quantidade de estruturas. A programação estruturada faz com que o programador precise ter uma organização em suas atividades, pois, com a disciplina exigida pela técnica, resultará em um sistema bem articulado. Veja a seguir um exemplo de um código, em português estruturado, utilizado dentro desta técnica:

```
[código]
algoritmo "media"
// Função: Calculo da media de um aluno exibindo
// se foi aprovado ou reprovado
// Autor: Manzano
// Data: 9/7/2005
// Seção de Declarações
var
resultado: caractere
n1, n2, n3, n4: real
soma, media: real
inicio
leia(n1, n2, n3, n4)
```



```
soma<- n1 + n2 + n3 + n4
media<- soma / 4
se(media >= 7) entao
  resultado<- "Aprovado"
senao
  resultado<- "Reprovado"
fimse
escreva("Nota 1: ", n1)
escreva("Nota 2: ", n2)
escreva("Nota 3: ", n3)
escreva("Nota 4: ", n4)
escreva("Soma: ", soma)
escreva("Media: ", media)
escreva("Resultado: ", resultado)
finalgoritmo
[/código]
```

A programação estruturada surgiu da necessidade de melhorar os processos de desenvolvimento de software, tornando-os mais eficazes. Diante de uma qualidade de códigos inferiores, uma produtividade ruim dos programadores e a ausência de um segmento de pesquisa mais formal, foi necessário buscar mecanismos que otimizassem o tempo destinado às atividades de programação. Esse modo de programar foi desenvolvido em relação a uma linguagem estruturada, também específica para a construção de códigos. Ela utiliza ainda estruturas de controle como sequência, condição e repetição, que permitem que cada bloco de código consiga ser compreendido de maneira mais detalhada. É fundamental que saibamos ainda que, neste processo de subdividir um método em “subproblemas”, na primeira divisão já é possível solucionar alguns, porém outros necessitarão ainda de outra subdivisão até que sua compreensão seja facilitada e a solução alcançada.

Tão importante quanto o reaproveitamento de softwares, a utilização desse tipo de programa permite ainda o uso dos tipos abstratos de dados, que serão aplicados na encapsulação daqueles dentro do sistema.

A ideia desta técnica está relacionada com a capacidade de reutilização de códigos e isso permite a otimização de tempo dentro de um processo de desenvolvimento de um programa. Nesse tipo de



programação, as linguagens utilizadas acoplam rotinas e estruturas de dados em um mesmo espaço, assim como também é possível desenvolver um objeto com características e atividades do objeto que o antecede. Veja a seguir a exemplificação:

Diagrama 2. Representação de uma programação orientada por objeto



Mas você deve estar se perguntando: como isso é possível? Em um exemplo prático, é viável executar uma rotina de uso comum, sendo essa a que tem condições de criar uma janela principal, com os indicativos do menu disponível. Caso fosse necessária a criação de outro aplicativo, somente era necessário passar algumas informações para a rotina de construção, sem ter que reescrever os mesmos comandos novamente.

Vale a ressalva de que a vantagem do uso dessa técnica está na capacidade de tornar o objeto passível de desenvolver atividades sem necessariamente precisar se atentar ao programa como um todo, olhando somente suas partes. Nesta programação, é comum o uso das linguagens Pascal, Java e C++, mas é preciso entender os aspectos que envolvem o seu uso.

A linguagem Pascal, por exemplo, você verá com frequência, pois ela é muito utilizada no meio acadêmico – justamente por ser bem estruturada. É considerada uma linguagem que precisa de um



compilador para converter o código-fonte em uma linguagem de máquina e permite a adoção de práticas fundamentais para o desenvolvimento do software. Para que você já se familiarize, veremos a seguir um exemplo de uma representação de um código em linguagem Pascal.

```
[código]
{Primeiro programa de exemplo em Pascal}
Program PrimeiroExemplo;
Var
  nota1, nota2, media: real;
  nome : string [30];
  resultado: string [10];
begin
  clrscr;
  writeln ('Digite o nome do aluno:');
  read ( nome);
  writeln('Digite a primeira nota: ');
  read ( nota1);
  writeln('Digite a segunda nota: ');
  read ( nota2);
  media:= (nota1 + nota2)/2;
  if (media >= 7) then
  begin
    resultado := 'aprovado';
  end
  else
  begin
    if(media >= 4) then
    begin
      resultado = 'emexame' ;
    end
    else
    begin
      resultado := 'reprovado';
    end
  end;
  writeln ('O aluno ' + nome + ' está ' + resultado);
  readkey;
end

[/código]
```



Perceba que os comandos para “início” e “fim” são utilizados para delimitar os blocos e são obrigatoriamente colocados em inglês, com o “*begin*” para início e “*end*” para fim. As manipulações de valores do tipo texto também são feitas utilizando aspas simples, diferente do pseudocódigo, que é feito com aspas duplas.

Ainda sobre a programação orientada, para que uma linguagem seja aplicada, precisa ter abstração, pois está lidando com a representação de um objeto em si e, nesse caso, faz-se necessário visualizar imaginariamente como este objeto irá se desenvolver dentro do sistema. Nessa abstração, precisamos levar em conta três pontos específicos: **identidade, propriedade e métodos**.

A identidade está relacionada à identificação que daremos ao objeto que será desenvolvido. Após a identidade, precisamos pensar nas características desse objeto, que, na programação orientada, conceituaremos como propriedades. Precisamos ainda delimitar quais serão as ações desse objeto, que nomearemos em programação como métodos. Vale ressaltar também que a programação orientada a objetos também é definida pela técnica de **encapsulamento**, que garante a segurança na programação, pois mantém as propriedades do objeto protegidas.

Na programação orientada por objetos, a reutilização de um código é uma importante vantagem, pois melhora a produção destinada à aplicação de um código. Essa herança é justamente o fato de herdar características de códigos antecessores, o que permitirá a otimização do sistema.

A depender da linguagem utilizada, a herança pode sofrer algumas alterações, inclusive fazer com que um mesmo código traga característica de mais de um antecessor ao mesmo tempo. É interessante sabermos também que, assim como acontece no ambiente da natureza em que algumas espécies são capazes de mudar a sua forma diante de uma necessidade, assim também acontece nesse tipo de programação, processo nomeado de **poliformismo**.

O poliformismo é o ato de alterar o funcionamento interno de um método herdado por um código “pai”. Justamente por ter essa característica é que precisamos nos atentar que nem sempre os métodos aplicados para a sua execução serão os mesmos. Haverá uma



variação a depender da linguagem utilizada e isso requer atenção para executar o método escolhido.

Processo de software

Diante do cenário que envolve tecnologia, é notório que a engenharia de software, a programação e os seus fundamentos estão em franca evolução ao longo das décadas. Diante desse cenário, nós vamos perceber como ocorre, de fato, o processo para criar um software, ou seja, uma série de atividades ligadas ao seu desenvolvimento.

Essa série de atividades permite o desenvolvimento de software por meio de uma linguagem de programação padronizada. Entretanto, vale fazer uma ressalva de que quando estas atividades forem indicadas para a realização de negócios, os métodos serão diferentes, pois os softwares de negócios são desenvolvidos por meio de extensão e manipulação de sistemas existentes.

Esses processos são indicados para diversas aplicações, porém todo o processo de criação precisa incluir algumas ações, como: a especificação de software, que estabelece sua funcionalidade e limitações; projeto e implementação de software, que aponta a criação e o desenvolvimento no sentido de suprir as especificações; validação de software para assegurar as demandas do cliente e o tempo de vida de software, que faz com que as necessidades de alteração provoquem a evolução dele. As atividades do processo de software são complexas, pois elas também executam tarefas secundárias, como a validação de requisitos. Além disso, elas são inseridas para ajudar no gerenciamento de configuração, por exemplo.

Como você pode perceber, o processo de software não é simples, pois demanda habilidade e capacidade intelectuais extremamente elevadas. Dessa forma, é primordial que as pessoas consigam tomar as suas decisões e realizar as suas análises. Diante dessa complexidade, não existe um processo ideal para uma determinada empresa, já que elas são responsáveis pelo próprio processo de desenvolvimento.



EXPLICANDO

A ideia de não haver o chamado “processo ideal” para o desenvolvimento de software não significa que as empresas devem parar de buscar a qualificação dos seus procedimentos, ou seja, a organização precisa adotar as técnicas mais recentes da engenharia de software.

Todo processo de software é representado por um modelo, que apresentará informações resumidas. Observando a engenharia de software, de maneira geral, existe um conjunto de modelos de processos que são formulados partindo de uma perspectiva ligada à sua arquitetura. Sendo assim, nós vamos discutir, especificamente, sobre o **modelo em cascata**, que abrange as tarefas básicas do processo. Posteriormente, trataremos o desenvolvimento **incremental**, que trata da relação existente entre essas atividades de processo. E, por fim, a **engenharia de software orientada a reuso**, que trata dos componentes reusáveis no desenvolvimento de um novo software.

O modelo em cascata

O primeiro modelo referente ao processo de desenvolvimento de software se caracteriza pelo encadeamento inserido nas fases do processo, sendo classificado como modelo em cascata. Inicialmente, o seu processo é orientado a planos, indicando a necessidade de um planejamento e a programação de todas as atividades do processo a serem realizadas. Esse modelo passa por estágios que indicam as atividades essenciais para o seu desenvolvimento. São elas:

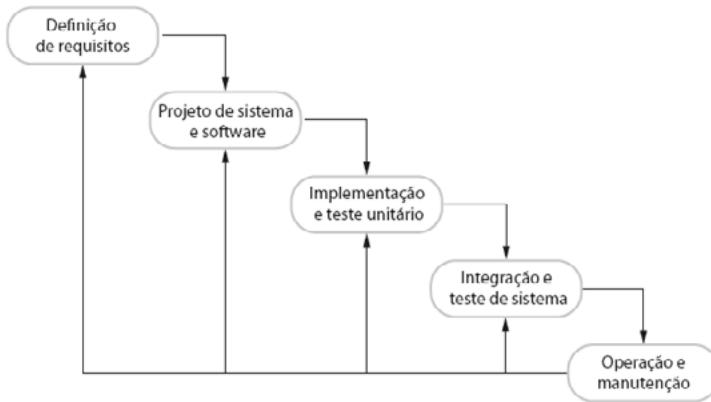
- **Análise e definição de requisitos:** nessa fase, as metas, restrições e serviços apresentados são instituídos por meio de consulta aos usuários.
- **Projeto de sistema e software:** por meio da arquitetura geral do sistema que o processo de projeto de sistemas vai adotar atributos para os sistemas.



- **Implementação e teste unitário:** fase em que o projeto do software é produzido conjuntamente com uma série de programas. Sendo assim, o teste unitário indica se cada unidade está respeitando a sua especificação estabelecida.
- **Integração e teste de sistema:** momento da integração e teste ocorrido nas unidades do programa com o objetivo de assegurar que os atributos sejam atendidos para posteriormente ficar à disposição do cliente.

Diagrama 3. Modelo cascata

O modelo em cascata



Fonte: SOMMERVILLE, 2011, p. 20.

- **Operação e manutenção:** considerada como a fase mais longa do ciclo de vida, a manutenção trata do conserto dos erros que não foram visualizados durante as fases iniciais do modelo.

Desenvolvimento incremental

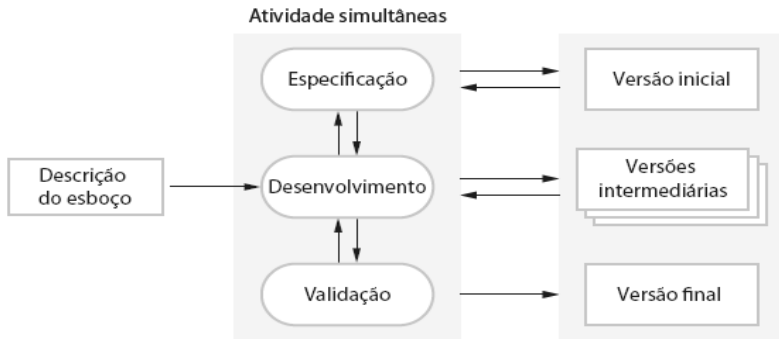
É um modelo de processo de software que almeja a implementação por meio da exibição aos comentários dos usuários e seguindo no desenvolvimento de diversas versões até o momento em que um sistema seja, de fato, concebido. As atividades referentes ao



processo (especificação, desenvolvimento e validação) vão se manter intercaladas entre si.

Diagrama 4. Representação do modelo de desenvolvimento incremental

Desenvolvimento incremental



Fonte: SOMMERVILLE, 2011, p. 22.

O desenvolvimento incremental de software é fundamental para as abordagens ágeis, diferenciando-se da abordagem em cascata no que se refere à maioria dos sistemas de negócios, por exemplo. Ele consegue solucionar problemas cotidianos. Trazendo esse conceito para o desenvolvimento de um software, torna-se menos custosa a realização de alterações no software.

Uma versão do sistema, assim como um incremento, pode adotar alguma funcionalidade fundamental para o cliente. Geralmente, os incrementos iniciais incluem os aspectos mais importantes. Nessa condição, o cliente consegue avaliar o desenvolvimento e verificar se os requisitos estão sendo atendidos. Importante frisar que o modelo incremental disponibiliza custos menores de acomodação às mudanças feitas nos requisitos do cliente, além de menores avaliações e documentação a ser refeita.

Outro aspecto é o fato de a relação estabelecida entre o cliente e desenvolvedor ser mais simples, gerando um retorno referente à qualidade do produto. A entrega de um software necessário ao cliente poderá ser realizada independente se a funcionalidade total ou não for acrescentada.



Vale ressaltar, porém, que o desenvolvimento incremental apresenta algumas desvantagens em relação ao seu gerenciamento. Falta visibilidade no processo, o que indica a necessidade de entregas periódicas, permitindo aos gestores a avaliação e evolução dos processos. Também existe uma relação inversa entre o desenvolvimento do sistema com a produção de documentos. As alterações constantes podem, de alguma forma, comprometer a sua estrutura, tornando complexas as futuras alterações ocorridas no software.

Engenharia de software orientada a reuso

Existe certa complexidade para reaproveitamento das características de um software na criação de um novo. Porém, boa parte dos projetos apresenta algum reuso de software, muitas vezes de maneira informal. As práticas do reuso informal são observadas independente do processo de desenvolvimento adotado. As suas abordagens precisam de uma base de elementos reusáveis de software, além de um framework que cuide da integração entre eles.

Importante frisar que a fase em que os requisitos iniciais são particularizados é comparável com a etapa da validação em relação a outros processos de software. Entretanto, quando se trata dos estágios mais intermediários direcionados a um processo orientado a reuso, fica visível a distinção. Tais estágios podem ser definidos como:

Quadro 1. Estágios de um processo orientado a reuso

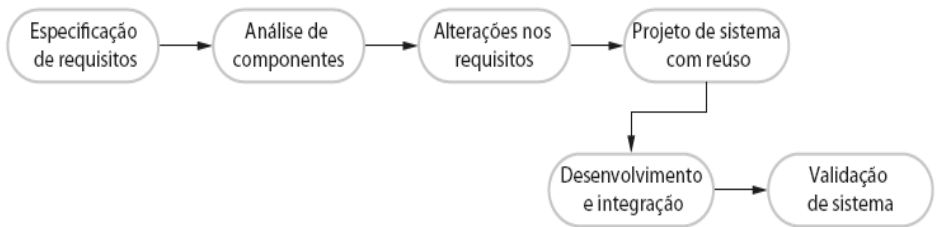
Análise de componentes	No momento em que a especificação de requisitos é definida, será feita uma pesquisa por componentes, visando a implementação desta particularização.
Modificação de requisitos	Nessa etapa os atributos são analisados através das informações referentes aos elementos visualizados. Depois eles serão alterados para apresentar os componentes disponíveis.
Projeto do sistema com reuso	Nessa fase, os desenvolvedores projetam o framework do sistema que será reutilizado. O objetivo é escolher os elementos que serão reusados e depois organizar o framework.



Desenvolvimento e integração	Existem casos em que os softwares não são adquiridos de maneira externa. Sendo assim, eles serão desenvolvidos pela própria empresa, com os elementos e sistemas sendo integrados e utilizados na criação de um novo sistema.
------------------------------	---

Diagrama 5. Organograma da aplicação da engenharia de software orientada a reuso

Engenharia de software orientada a reuso



Fonte: SOMMERVILLE, 2011, p. 20.

Dentre os modelos de componentes de software adotados, é possível destacar os **web services**, que são criados com base nos padrões de serviço e estão disponíveis para a chamada **invocação remota**. Outro modelo é a coleção de objetos criada com o objetivo de ser integrada com um framework de elementos. Por fim, os sistemas de software (stand-alone), que passam por alterações visando o seu uso em determinados ambientes.

A engenharia de software orientada a reuso visa reduzir a quantidade de software que é criado e, consequentemente, limitar os custos. Esta ação intenciona promover uma entrega mais rápida do software sem abrir mão dos requisitos do sistema.



Linguagem de programação

A engenharia de software expõe como finalidade principal traduzir ou simplificar as representações pertencentes a um software de uma maneira que seja compreendida pelo computador. Entretanto, nós vamos tratar aqui sobre outro aspecto essencial: a **codificação**, que consiste, basicamente, no processo de transformar o projeto desenvolvido em uma linguagem de computação.

Existe certa complexidade no desenvolvimento da linguagem de programação por se tratar de uma codificação específica. Diante desse cenário, existe uma corrente entre usuário e programadores para que este quadro se altere. Só pra citar um exemplo, as requisições de serviço de processamento de uma máquina podem ser feitas em uma linguagem mais corriqueira, como o inglês. Veremos mais adiante que, ao invés de simplesmente codificar, os desenvolvedores de algumas classes conseguem descrever os resultados previstos do procedimento almejado. Nessa condição, o código-fonte pertencente a uma linguagem de programação convencional será criado de maneira automática.

Importante frisar que a codificação é conceituada como uma consequência que o projeto está submetido. Entretanto, os aspectos da linguagem de programação e o estilo de codificação adotado podem interferir, de forma significativa, nos padrões de qualidade e manutenção do software. Em paralelo a isso, temos o processo de **tradução**, que é uma ferramenta em que o programa é transformado de uma linguagem de programação anterior para uma mais atual, com a mesma linguagem ou diferente. Importante lembrar que a saída pertencente ao compilador é convertida, de maneira adicional, dentro de um código de máquina, que são instruções reais responsáveis pelo acionamento da lógica de maneira microcodificada na unidade centralizada de processamento.

Dentro da engenharia de software, existe uma preocupação imediata denominada de **tradução inicial**, que começa no projeto detalhado à linguagem de programação. Importante deixar claro que uma interpretação errada de determinada especificação de um planejamento de detalhes normalmente vai ocasionar um



código-fonte equivocado, ou seja, algumas complexidades ou limitações na linguagem de programação podem gerar um código-fonte com alto nível de distorção, com manutenção e testes difíceis.

Características das linguagens de programação

Bem, o que seriam de fato as linguagens de programação? Elas são entendidas como um meio de comunicação estabelecido entre as pessoas e os computadores, lembrando que a codificação é vista como uma atividade exercida pelo ser humano. Dentro desse contexto, os aspectos psicológicos causam um impacto interessante em relação ao nível de qualidade da comunicação adotada. Importante deixar claro que o processo de codificação é essencial aos procedimentos realizados na engenharia de software. Com isso, os aspectos da engenharia de uma linguagem conseguem impactar sobre o projeto de criação e na qualidade.

Trazendo uma visão psicológica referente ao software, é preciso ter em mente que o seu principal objetivo é focar nas preocupações que os seres humanos têm em relação à facilidade, aprendizagem simples e confiabilidade em paralelo com o nível de eficiência da máquina, capacidade apresentada pelo software. Apesar da utilização de métodos de análise e projetos apoiados em ferramentas automatizadas (CASE) que conseguem disponibilizar um suporte substancial, é preciso entender que a engenharia do software é vista como uma tarefa exercida por seres humanos. Vale frisar que um conjunto de aspectos psicológicos vão se apresentar como resultados oriundos dos projetos de linguagem de programação. Além das características serem passíveis de medição, vamos reconhecer suas manifestações inerentes nas linguagens de programação.

A primeira característica é a **uniformidade**, que aponta o nível em que uma linguagem adota uma notação mais consistente, insere limitações mais arbitrárias e do suporte às exceções mais sintáticas. Outro aspecto trata da **ambiguidade**, que normalmente é notada pelo programador. Nessa situação, um compilador vai entender



uma instrução de uma determinada maneira, enquanto o leitor humano visualizará esta mesma instrução de maneira distinta.

Quando tratamos da concisão de uma linguagem de programação, essa representa o apontamento de informações direcionadas para o código que serão recuperadas da memória humana, de novo. Nesse tipo de aspecto, é possível encontrar, por exemplo, o nível a que a linguagem consegue suportar o desenvolvimento das estruturas; os modelos de palavras-chave que podem ser utilizadas, a diversidade de dados e a quantidade de números de operadores lógicos e funções embutidas.

A localidade é uma característica que se amplia no momento em que as declarações conseguem ser reunidas em blocos quando as estruturas são introduzidas de maneira direta e os projetos e os códigos são coesos e **modulares**.

A linearidade é um aspecto ligado ao entendimento sobre a manutenção do domínio funcional. E o que isto quer dizer? Bem, isso indica que a percepção humana fica mais simplificada no momento em que uma sequência de operações lógicas é visualizada. Existe uma ramificação mais extensa que altera o processo de linearidade existente. Nessa condição, a implantação das estruturas ocorridas de maneira direta vai ajudar na linearidade da linguagem de programação.

Por fim, temos a **tradição**, que trata da capacidade de compreender uma linguagem de programação. Esse tipo de aspecto interfere no nível de inovação ao longo de uma nova linguagem. É importante frisar que os aspectos psicológicos pertencentes à linguagem de programação estabelecem uma relação essencial com a nossa capacidade de aprendizado, aplicação e manutenção. Entretanto, é bom deixar claro que uma linguagem de programação ilustra a forma como idealizamos os programas e restringe como nos comunicamos com a máquina.

Outro aspecto importante que devemos tratar aqui se refere ao modelo sintático/semântico. Do que se trata? Bem, esse modelo tem uma importância grande quando estivermos na fase de codificação. No momento em que o programador insere métodos de engenharia de software, o conhecimento semântico será inserido,



até porque esse conceito é independente. Por sua vez, o conhecimento sintático estará relacionado à linguagem de programação em si e irá se concentrar nos aspectos pertencentes a uma linguagem específica.

No momento em que o usuário consegue compreender uma nova linguagem de programação, a memória irá armazenar novas informações sintáticas. Isso pode ser visto como complexo no momento em que percebermos que uma nova linguagem for parecida, porém não será equivalente à sintaxe pertencente à outra linguagem, mas é bom deixar claro que este tipo de situação pode, de certa maneira, “obrigar” o engenheiro de software a compreender as novas informações semânticas.



EXPLICANDO

No momento em que alguns argumentos referentes à necessidade de criação de código de fato aparecerem, é possível notar que determinados problemas de software não ocorrem por conta da falta de conhecimento da parte sintática do modelo, mas sim do conhecimento semântico, ligado à nossa capacidade de aplicação.

Falando sobre a visão da engenharia, é preciso ter em mente que a linguagem se concentra nas demandas de um projeto de criação de um software específico. Nesse contexto, é possível estabelecer uma série de características que buscam, dentre outros aspectos, simplificar a conversão de um projeto em código, verificar o nível de eficiência de um compilador, realizar a portabilidade do código-fonte, disponibilizar as ferramentas de criação e realizar as práticas de manutenibilidade. Veremos cada um destes aspectos:

- **A simplificação da transformação de um projeto em código:** visa disponibilizar, de maneira estreita, como uma linguagem de programação simboliza um projeto.



- **Eficiência do compilador:** trata de uma das exigências da engenharia, pois os avanços na velocidade dos processadores e na densidade de memória foram iniciados para mitigar a necessidade de “códigos supereficientes”, sendo que muitas aplicações ainda requerem programas rápidos e concisos (baixo requisito de memória).
- **Portabilidade de código:** trata-se de um aspecto que pode ser transferido entre processadores e compiladores com baixa modificação por meio de um código-fonte imutável, mesmo quando o ambiente se altera, ou por meio de um código-fonte integrado em pacotes distintos de softwares com baixa modificação.
- **Disponibilidade de ferramentas de desenvolvimento:** responsável por simplificar o tempo necessário para criar o código-fonte e ampliar o nível de qualidade.
- **Manutenibilidade do código-fonte:** não pode ser realizada enquanto não houver compreensão total do software. Os componentes anteriores pertencentes à configuração de software são base para esta compreensão, porém o código-fonte será alterado e visto em paralelo com as alterações no projeto.

Não podemos esquecer que a escolha de uma linguagem de programação precisa levar em consideração os aspectos psicológicos e os de engenharia. Entretanto, o problema ligado a essa escolha deve ser analisado caso uma linguagem esteja disponível ou apresentada por algum solicitante.

A linguagem de programação vai exercer um impacto sobre todas as fases pertencentes à execução de um projeto, desde o planejamento até a manutenção. Ela disponibiliza maneiras de traduzir o processo entre o homem e a máquina, porém, a qualidade do resultado final está relacionada às tarefas desenvolvidas pela engenharia de software, que segue a codificação.



Aspectos fundamentais das linguagens de programação

Os aspectos técnicos referentes às linguagens de programação reúnem uma quantidade extensiva de tópicos que podem variar da especificação de uma linguagem mais formalizada a comparações funcionais de linguagens mais específicas. Para que possamos compreender essas características fundamentais, é preciso visualizar quatro temas de maneira ampliada: **tipo de dados, mecanismos de subprograma, estrutura de controle e suporte para abordagens orientadas a objeto.**

As linguagens de programação apresentadas estão ligadas diretamente a esses tópicos e sua qualidade pode ser analisada levando em consideração as potencialidades e fragilidades. Veremos com mais detalhes a seguir.

Tipologia dos dados

Atualmente, a sintaxe e a amplitude do desenvolvimento de linguagens de programação sozinhas são insuficientes para analisar a sua qualidade. Nesse contexto, é preciso levar em consideração os tipos de dados específicos que uma linguagem de programação suporta e que são vistos como o aspecto necessário para verificar a qualidade inserida na linguagem.

Mas o que seria de fato essa tipologia de dados? Bem, trata-se de uma classe de objetos de dados aliados a uma série de operações para criar e manipular dados. Importante deixar claro que um objeto de dados visa assumir um determinado valor que se encontra dentro dos limites indicados para cada modalidade, além da possível manipulação por meio das operações inseridas.

É preciso ter em mente que a tipologia dos dados consegue ser inserida em um limite amplo, que vai abranger conjuntos numéricos, booleanos e da string. Vale ressaltar que tipos de dados com maior nível de complexidade conseguem reunir uma estrutura capaz de percorrer um simples vetor até uma estrutura listada, por exemplo.



Vamos levar em consideração o conceito definido por Fairley (1985) apud Pressman (1995, p. 690), que estabelece alguns níveis de verificação dos modelos geralmente observados nas linguagens de programação. Vamos analisá-los de maneira individualizada:

- **Nível 0 (zero):** as chamadas linguagens de programação “sem tipo” não apresentam nenhuma forma explícita de tipologia e não realizam a verificação desses tipos. A linguagem *basic* é um dos exemplos dessa categoria. Vale lembrar que, além da possibilidade do usuário de estabelecer a estrutura de dados, a sua representação será determinada previamente e estará contida em cada objeto de dados.
- **Nível 1 (um):** consiste em uma técnica de verificação que possibilita que o programador reúna dados distintos, mas que, posteriormente, consiga converter os operandos que apresentam modelos não compatíveis, o que permite que as operações requisitadas ocorram.
- **Nível 2 (dois):** trata-se de uma conversão mista de tipos e se caracteriza por apresentar modelos distintos de dados dentro de uma mesma categoria para que sejam transformados em um único modelo de destino.
- **Nível 3 (três):** trata-se da verificação de tipos que apresentam características da verificação “forte” por oferecer “furos” na sua execução, ou seja, a linguagem Pascal, por exemplo, não consegue compatibilizar as interfaces em módulos separadamente compilados.
- **Nível 4 (quatro):** trata-se da verificação forte de tipos que se caracterizam por acontecer em linguagens que só permitem que as operações sejam do mesmo modelo de dados especificados de maneira prévia.

Subprograma

Outro aspecto que devemos mencionar se refere aos subprogramas, um elemento pertencente a um programa compilável de maneira separada, que apresenta uma estrutura de controle e de dados.



Independente da nomenclatura adotada, um subprograma se caracteriza por apresentar uma série de aspectos gerais, como:

1. A seção que especifica a nomenclatura e descrição da interface;
2. A seção de instalação, incluindo a estrutura de controle e de dados;
3. Mecanismo de ativação que permite que o subprograma seja chamado de qualquer localidade do programa.

Um detalhe importante que devemos mencionar é que as linguagens de programação mais convencionais, os **subprogramas**, de forma individualizada, são consideradas como entidades em si, atuando com dados guiados pela estrutura de controle inserida em um programa mais extenso. Importante ressaltar que, nas linguagens de programação consideradas orientadas a objeto, o objeto substitui a ideia clássica pertencente ao subprograma.

Estruturas de controle

Fundamentalmente, as linguagens de programação consideradas como “modernas” permitem ao programador simbolizar as construções lógicas pertencentes a uma programação estruturada. Boa parte destas linguagens disponibiliza uma sintaxe que trata das especificações diretas. Lembrando que outras modalidades de linguagem conseguem exigir que o programador desenvolva uma emulação das estruturas respeitando os limites mais sintáticos pertencentes à linguagem.

O que você precisa entender é que, além dos procedimentos essenciais de programação estruturada, é possível verificar a presença de outras estruturas de controle. Falando primeiramente da recursão, iremos notar que ela desenvolve uma segunda ativação pertencente a um subprograma ao longo da primeira ativação, fazendo com o que subprograma seja ativado como uma etapa do procedimento estabelecido.

Outro aspecto se refere à concorrência, que disponibiliza auxílio no desenvolvimento de diversas atividades para a comunicação geral e sincronização entre as atividades. Lembrando que este



aspecto de linguagem é essencial quando as aplicações de sistema são realizadas. Por fim, podemos citar o chamado **manejo de exceções**, que se apresenta como uma linguagem de programação que busca possíveis erros no sistema e transfere para o controle de um manipulador que realiza o processamento.

Abordagens orientadas a objetos

Desenvolver objetos e softwares orientados a objeto pode gerar algum efeito na utilização de linguagem de programação qualquer. Entretanto, o que podemos observar na prática é que o auxílio direcionado às abordagens orientadas precisa estar inserido nas linguagens de programação de forma direta, com o objetivo de implementar um projeto orientado a objeto.

Uma linguagem de programação orientada a objetos, por característica, precisa disponibilizar um auxílio direto para estabelecer algumas classes ou encapsulação, por exemplo. Além disso, outras linguagens com essas mesmas características implementam aspectos adicionais.

É relevante afirmar que a determinação de classes é essencial quando tratamos de uma abordagem orientada a objetos, pois a sua linguagem estabelece uma nomenclatura para a classe além do elemento que a compõe. Para ilustrar um exemplo desses modelos de abordagem, vamos utilizar uma classe denominada de **conter**, que se encontra em uma linguagem C.

[código]

```
Class conter
{
Private:
    Unsignedintvalue

Public
    Conter ();
    Voidincrement ();
    Voidincrement ();
    Unsigned intacess_value ();
};
```

Diante deste contexto, uma classe pode ser extraída de outra básica por meio da seguinte codificação:



```
Class special_counter: public counter
{
    Uma cópia dos dados privados de "counter"
Private:
    Data privatespecial_counter
...
};
```

[/código]

Quais informações podem ser extraídas dessa codificação? Bem, os objetos oriundos da classe *Special_counter* utilizam as técnicas estabelecidas para a classe-mãe *counter*. Ao estabelecer uma determinada classe, as abstrações de dados e os elementos pertencentes ao programa que operam neles serão encapsulados.

Classes de linguagem

Diversas linguagens de programação normalmente são adotadas para auxiliar na criação de um software. Diante disso, existe um amplo debate sobre as categorias que esta programação pertence. Vamos visualizar a evolução histórica dessas gerações de linguagens?

Linguagens de primeira geração: trata-se de uma modalidade de linguagem em que a codificação ocorre em nível de máquina, presente nas aplicações até os dias atuais. Só para se ter um exemplo, a Assembly simboliza a primeira geração de linguagens de programação. Importante deixar claro que as linguagens que são dependentes da máquina apresentam um grau baixo de abstração.



EXPLICANDO

As arquiteturas de processadores que utilizam as chamadas instruções customizadas são exemplos de linguagens Assembly. Do ponto de vista de um profissional da área de software, essas linguagens podem ser empregadas no momento em que uma de alto nível não conseguir suprir os requisitos necessários.



Linguagens de segunda geração: são linguagens que surgiram entre o final dos anos 50 e início dos anos 60, sendo consideradas como a base para as modernas. Essa modalidade se caracteriza, dentre outros aspectos, pelo uso ampliado, por adotar bibliotecas de software de maneira extensiva com alto índice de aceitação.

Podemos citar algumas linguagens que simbolizam esta fase do desenvolvimento. São elas:

- **FORTRAN:** considerada como a primeira linguagem a ser desenvolvida para a área de engenharia, caracterizou-se por resolver problemas computacionais. Entretanto, a sua deficiência estava no auxílio direto às estruturas construídas, pois apresentava uma tipologia de dados limitada. Depois de algumas evoluções, ela passou a ser utilizada em aplicações para softwares mais básicos.
- **COBOL:** bastante utilizada em aplicações de dados comerciais, apresenta certa deficiência no que diz respeito à concisão, porém disponibiliza uma capacidade elevada de definição de dados, autodocumentação e suporte a um número elevado de métodos ligados ao processamento de dados comerciais.
- **ALGOL:** considerada como uma das precursoras das linguagens de terceira geração, disponibiliza uma vasta quantidade de construções procedimentais e tipos de dados. Caracteriza-se por trazer uma noção de estruturação e alocação de memória mais dinâmica e recursão.
- **BASIC:** idealizada para trazer conhecimento sobre programação em modo compartilhado de tempo. Passou por uma reformulação ao final dos anos 70 e por conta da sua diversidade de versão existe uma ampla discussão sobre as vantagens e deficiências desta linguagem.

Linguagens de terceira geração: são caracterizadas por apresentar uma capacidade elevada de estruturação tanto na parte procedimental quanto na de dados. Podemos subdividi-las basicamente em três categorias mais ampliadas. São elas:



- **Linguagens de alto nível de uso geral:** o ALGOL, que é considerada a linguagem de alto padrão mais antiga, caracteriza-se por servir de pauta para outras linguagens, como o Pascal, Modula - 2, por exemplo. Essas linguagens apresentam uma capacidade ampla de serem utilizadas em diversas áreas relacionadas; por exemplo, a engenharia/científica, produtos comerciais, dentre outros.
- **Linguagens orientadas a objeto:** possibilitam ao profissional da engenharia de software a implementação de modelos de análise e projetos desenvolvidos pela OOA e OOD. As linguagens orientadas a objeto já vêm sendo introduzidas há um bom tempo, porém algumas ganharam destaque no meio corporativo, como os dialetos da linguagem C (como o C++ e o Smalltalk e o Eiffel, bastante utilizado em aplicações industriais).
- **Linguagens especializadas:** são formas sintáticas projetadas especialmente para uma aplicação diferente. Existe uma diversidade de linguagens deste tipo que ainda é utilizada até os dias atuais. Dentre as linguagens mais usadas, podemos citar a LISP, que é empregada na manipulação de símbolos; o PROLOG, que simboliza o conhecimento, servindo de base para uma estrutura de dados uniforme; o APL, que é bastante utilizado na manipulação de vetores e arrays e a FORTH, que foi projetada para a criação de softwares em microprocessadores.

Se considerarmos os conceitos abordados pela engenharia de software, as linguagens especializadas irão disponibilizar uma série de vantagens. A transformação dos requisitos em projetos para implementação pode ser simplificada caso a linguagem especializada seja criada para cuidar de uma aplicação específica. Por sua vez, boa parte das linguagens especializadas apresenta portabilidade menor, o que reduz a capacidade de obter manutenção.

Linguagens de quarta geração: durante a o período histórico de desenvolvimento de software, os profissionais da área de criação buscam inserir programas com índices de abstração cada vez mais altos. Isso é perceptível quando analisamos as chamadas linguagens de programação de quarta geração. Vale frisar que essas linguagens,



assim como as outras artificiais, disponibilizam uma sintaxe diferente para simbolizar a estrutura de dados e controle.

As linguagens de quarta geração reúnem aspectos classificados como **procedimentais e não procedimentais**. Isso significa que a linguagem permite ao usuário determinar condições e ações correspondentes ao mesmo tempo em que aponta sobre o resultado almejado para inserir um conhecimento específico. As linguagens podem ser classificadas em:

- **Linguagem de consulta:** as linguagens 4GLs foram projetadas para ser aplicadas agrupadas no banco de dados. Elas permitem ao usuário a manipulação sofisticada de informações contidas nos banco de dados.
- **Geradores de programas:** simbolizam a classe mais sofisticada na linguagem de quarta geração. Essa modalidade se caracteriza por permitir que o usuário desenvolva programas de linguagem utilizando declarações.
- Outras 4GLs: são as categorias de linguagens de quarta geração normalmente utilizadas. Dentre as mais importantes, podemos citar as linguagens de apoio e decisão, que permitem aos não programadores variar simples modelos de planilhas até modelagens de estatísticas mais sofisticadas. As linguagens de prototipação foram criadas para ajudar no desenvolvimento de protótipos e as formais de especificação geram softwares executáveis dentro da máquina.



Estilo de codificação

Depois que o código-fonte é criado, o objetivo do módulo precisa ser especificado de maneira clara, ou seja, o código precisa se mostrar de forma inteligível, pois descrever um programa de maneira bem simplista é redigir uma sequência de declarações presentes na linguagem disponível. Dentre os elementos que compõem o estilo de codificação, podemos incluir:

- **Documentação interna do código-fonte:** começa com a seleção de nomes denominados de **identificadores**, continua por meio da alocação e formação dos comentários e termina por meio da organização visual do programa.

Diante desse aspecto, é preciso fazer uma ressalva: a escolha de nomes significativos é essencial para que exista uma compreensão mais clara. Às vezes, a simplificação da nomenclatura acaba deixando o seu significado obscuro. Observe a declaração a seguir:

$D = V * T$

DIST = VELHOR 8 TEMP

DISTÂNCIA = VELOCIDADE_ HORIZONTAL * TEMPO_ PERCORRIDO_ EM_ SEGS;

As três declarações foram feitas, respectivamente, em linguagens BASIC, FORTRAN e ALGOL. Logicamente, a mais detalhada foi a última, pois apresenta de maneira clara como o cálculo será, de fato, realizado.

- **Declaração de dados:** é preciso ter em mente que a complexidade e a organização da estrutura de dados são estabelecidas ao longo do projeto. O estilo da declaração de dados é formado no momento em que o código é criado. Um conjunto de diretrizes precisa ser simples e será definido para tornar compreensíveis os dados e a sua manutenção.



Importante frisar que a declaração de dados apresenta uma maneira padronizada, apesar da linguagem de programação não disponibilizar os requisitos considerados como obrigatórios. Observe o módulo FORTRAN a seguir:

1. **Declaração explícita:** INTEGER, REAL.
2. **Blocos de dados globais:** COMMON.
3. **Arrays locais:** DIMENSION.
4. **Declarações de arquivos:** DEFINE FILE.

A ordem como as declarações são expostas simplifica os atributos, tornando as atividades de testes mais ágeis:

- **Construção de instruções:** é definida ao longo da etapa de projeto, porém, essa construção está inserida na fase de codificação. A construção de instruções segue uma norma específica: elas precisam ser simplificadas, ou seja, não interferir na eficiência.
- **Entrada/saída:** é estabelecida no momento da fase de avaliação dos requisitos de software e projeto, exceto o de codificação. Entretanto, a forma como esse estilo é inserido pode ser o aspecto essencial para a aceitação do sistema por conta dos usuários.

Vale a pena trazer outra visão sobre o tema ao tratar especificamente da linguagem Python, que é conhecida como uma linguagem do tipo funcional impura, ou seja, ela tem a capacidade de suportar diversos paradigmas presentes na programação. A depender da situação, usar esta modalidade de linguagem significa obter uma vantagem ou desvantagem. Diante desse cenário, há quatro aspectos (paradigmas), também conhecidos como estilos de codificação, para demonstrar as essencialidades do Python:



- **Funcional:** considera que toda instrução consiste em uma equação matemática. É um estilo bastante utilizado para realizar tarefas de processamento em paralelo.
- **Imperativo:** as alterações vão acontecer de acordo com as mudanças presentes no estado do programa. Importante ressaltar que este estilo visa, dentre outros aspectos, manipular as estruturas de dados.
- **Orientado a objetos:** utilizado com outras linguagens, visa deixar mais simples o ambiente de codificação para criar um modelo para o mundo real.
- **Procedural:** é um estilo adotado para criar uma iteração e aspectos ligados ao sequenciamento, seleção e modularização, além de se apresentar como a maneira mais simples que a codificação pode utilizar.



SINTETIZANDO

Vimos, nesta unidade, que a programação pode ser entendida como o processo para desenvolver um programa de computador, o que envolve a escrita e a manutenção de um programa, com linguagem específica. Estudamos sobre as técnicas de desenvolvimento de programas, sendo as principais para modularizar o código a top-down e a bottom-up. Também aprendemos que a programação estruturada é definida pela utilização de blocos como estrutura básica para a construção de um sistema e pudemos analisar um exemplo de código. Dentro da programação orientada, verificamos a necessidade de abstração, que deve levar em conta três pontos específicos: identidade, propriedade e métodos.

Estudamos o modelo em cascata, que abrange as tarefas básicas do processo, o desenvolvimento incremental, que trata da relação existente entre estas atividades de processo e, por fim, a engenharia



de software orientada a reuso, que trata dos componentes reusáveis no desenvolvimento de um novo software.

Observamos que a engenharia de software simplifica as representações de forma que sejam compreendidas pela máquina. Nesse contexto, vimos que a codificação é importante para transformar o projeto desenvolvido em uma linguagem de computação. Falando sobre os aspectos fundamentais das linguagens de programação, compreendemos que podem ser divididos em quatro grandes temas: tipo de dados, mecanismos de subprograma, estrutura de controle e suporte para abordagens orientadas a objeto.

Após esses estudos, o aluno será capaz de identificar as classes de linguagem (primeira, segunda, terceira e quarta gerações), bem como estilos diferentes de codificação. Vimos que, ao invés de simplesmente codificar, os desenvolvedores de algumas classes conseguem apresentar os resultados esperados por meio do procedimento almejado.



Referências bibliográficas

Unidade 1

ALVES, W. P. **Lógica de programação de computadores**: ensino didático. São Paulo: Érica, 2010.

_____. **Linguagem e lógica de programação**. São Paulo: Érica, 2014.

BENEDUZZI, H. M.; METZ, J. A. **Lógica e linguagem de programação: introdução ao desenvolvimento de software**. Curitiba: LT, 2010.

O QUE É e como funcionam os algoritmos?. Postado por Pesquisa Fapesp. (6 min. 58 s.). son. color. port. Disponível em: <https://www.youtube.com/watch?v=Xo1V_JL1yAg>. Acesso em: 16 jan. 2019.

SOFFNER, R. **Algoritmos e programação em linguagem C**. São Paulo: Saraiva, 2013.

Unidade 2

ALVES, W. P. **Linguagem e lógica de programação**. 1. ed. São Paulo: Érica, 2014.

COCIAN, L. F. E. **Manual da linguagem C**. Canoas: Ed. ULBRA, 2004.

MELO, D. T. **Lógica de programação com Visualg** - uma abordagem prática. São Paulo: Ed. Do Autor, 2013.

PASCHOALINI, G. R. **Princípios da lógica de programação**. São Paulo: Senai Editora, 2017.

QUIERELLI, D. A. **Algoritmos e lógica de programação em pseudódigos, linguagem C e Java**. 1. ed. São Paulo: Leme, 2012.



SOFNER, R. **Algoritmos e programação em linguagem C**. 1. ed. São Paulo: Saraiva, 2013.

Unidade 3

BENEDUZZI, H. M.; METZ, J. A. **Lógica e linguagem de programação**: introdução ao desenvolvimento de software. Curitiba: Editora do Livro Técnico, 2010.

MANZANO, J. A. N. G. **Estudo dirigido**: algoritmos. São Paulo, SP: Érica, 2000.

SOFFNER, R. **Algoritmos e programação em linguagem C**. São Paulo, SP: Saraiva. 2013.

VISUALG 3 – download. Postado por GuiaTech. (5min. 52s.). son. color. port. Disponível em: <<https://youtu.be/wAH3gPMW2lo>>. Acesso em: 11 fev. 2020.

Unidade 4

ALVES, W. P. **Linguagem e lógica de programação**. 1. ed. São Paulo: Érica, 2014.

BENEDUZZI, H. M.; METZ, J. A. **Lógica e linguagem de programação**: introdução ao desenvolvimento de software. Curitiba: Livro Técnico, 2010.

MANZANO, J. A. N. G. **Estudo dirigido**: algoritmos. São Paulo: Érica, 2000.

MUELLER, J. P. Programação funcional para leigos. Rio de Janeiro: Alta Books, 2019.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de software**. Porto Alegre: AMGH, 2016.